Bài 1.5 – Matplotlib/Seaborn: vẽ biểu đồ để "nhìn" dữ liệu

> **Giai đoạn 1 – Python cho ML & Data Science** · Thời lượng: 3 buổi · Hình thức mở đầu: *Đố vui trước khi học*

Chia buổi:

- **Buổi 1:** Mục 1 → 6.7 (vì sao phải vẽ, cấu tạo biểu đồ, 6 loại biểu đồ cơ bản với Matplotlib).
- **Buổi 2:** Mục 6.8 → 6.16 (nhiều biểu đồ trên một hình, tùy chỉnh, Seaborn cho phân bố, nhóm, quan hệ).
- **Buổi 3:** Mục 6.17 → hết (heatmap, pairplot, bản đồ, kiểm chứng giả thuyết, lỗi thường gặp, trắc nghiệm, thực hành).

---

1. 🎯 Mục tiêu bài học

Sau bài này bạn có thể...

- Giải thích vì sao **không bao giờ** chỉ nhìn con số thống kê mà bỏ qua biểu đồ.
- Chọn đúng loại biểu đồ cho từng câu hỏi: phân bố, so sánh nhóm, quan hệ, xu hướng theo thời gian, tương quan.
- Vẽ và tùy chỉnh line, scatter, bar, histogram, boxplot bằng Matplotlib theo kiểu `fig, ax`; ghép nhiều biểu đồ trên một hình.
- Dùng Seaborn để vẽ nhanh biểu đồ thống kê với `hue`, `col`: histplot, boxplot, violinplot, barplot, scatterplot, lineplot, regplot, heatmap, pairplot.
- Đọc biểu đồ để phát hiện vấn đề dữ liệu (giới hạn trần, outlier, phân bố lệch) và kiểm chứng giả thuyết, tránh các biểu đồ gây hiểu lầm.

---

2. 🔗 Ôn nhanh kiến thức cần có

- **NumPy** (bài 1.3): `np.linspace` tạo điểm để vẽ đường cong; `np.arange` tạo vị trí cột.
- **Pandas** (bài 1.4): `groupby`, `value_counts`, `pd.cut`, `pivot_table`, `corr()` tạo ra bảng số để vẽ.
- **Dạng dài** (bài 1.4, `melt`): mỗi dòng một quan sát. Seaborn thích dữ liệu dạng này.
- **Giả thuyết bỏ ngỏ** ở bài 1.4: "*Vì sao khu nhà Cũ lại đắt hơn?*" Hôm nay ta trả lời bằng hình.

---

3. 🧩 Mở đầu: Đố vui trước khi học 🤔

Có 4 bộ dữ liệu A, B, C, D, mỗi bộ có 11 điểm (x, y). Bạn tính thống kê và được:

```
A  TB x = 9.0 | TB y = 7.5 | ĐLC y = 2.03 | tương quan = 0.816
B  TB x = 9.0 | TB y = 7.5 | ĐLC y = 2.03 | tương quan = 0.816
C  TB x = 9.0 | TB y = 7.5 | ĐLC y = 2.03 | tương quan = 0.816
D  TB x = 9.0 | TB y = 7.5 | ĐLC y = 2.03 | tương quan = 0.817
```

(TB = trung bình, ĐLC = độ lệch chuẩn. Tương quan đo mức hai biến "đi cùng nhau", sẽ học kỹ ở bài 1.7.)

Câu hỏi: 4 bộ dữ liệu này có giống nhau không? Hãy đoán trước khi chạy code.

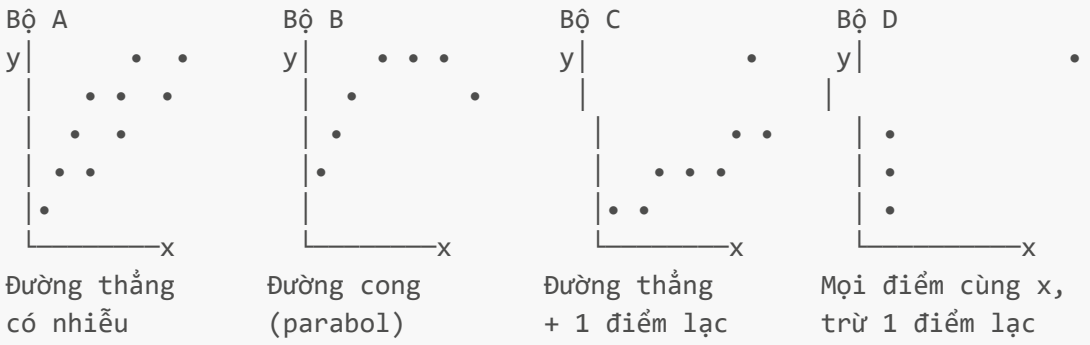
Hầu hết mọi người đoán "gần như giống hệt". Giờ hãy vẽ:

```
import matplotlib.pyplot as plt   # thư viện vẽ cơ bản, quy ước tên tắt là plt

# Bộ tứ Anscombe (Anscombe's quartet), một ví dụ kinh điển năm 1973
x  = [10, 8, 13, 9, 11, 14, 6, 4, 12, 7, 5]
y1 = [8.04, 6.95, 7.58, 8.81, 8.33, 9.96, 7.24, 4.26, 10.84, 4.82, 5.68]
y2 = [9.14, 8.14, 8.74, 8.77, 9.26, 8.10, 6.13, 3.10, 9.13, 7.26, 4.74]
y3 = [7.46, 6.77, 12.74, 7.11, 7.81, 8.84, 6.08, 5.39, 8.15, 6.42, 5.73]
x4 = [8, 8, 8, 8, 8, 8, 8, 19, 8, 8, 8]
y4 = [6.58, 5.76, 7.71, 8.84, 8.47, 7.04, 5.25, 12.50, 5.56, 7.91, 6.89]
bo = {"A": (x, y1), "B": (x, y2), "C": (x, y3), "D": (x4, y4)}

fig, axes = plt.subplots(1, 4, figsize=(16, 4))     # 1 hàng, 4 ô biểu đồ
for ax, (ten, (xs, ys)) in zip(axes, bo.items()):   # zip: ghép từng ô với từng bộ
dữ liệu
    ax.scatter(xs, ys)                               # vẽ điểm
    ax.set_title(f"Bộ {ten}")
plt.tight_layout()                                   # tự căn chỉnh để chữ không
đè nhau
plt.show()
```

Bạn sẽ thấy 4 hình hoàn toàn khác nhau:



Bài học: Con số thống kê **giống hệt nhau** có thể che giấu những bức tranh **hoàn toàn khác nhau**. Một mô hình đường thẳng (bài 2.2) chỉ hợp với bộ A. Vì vậy, quy tắc số 1 của người làm ML là: **luôn vẽ dữ liệu trước khi làm gì khác**.

---

4. 💡 Trực giác

4.1. Bảng số là "bản nhạc", biểu đồ là "bài hát"

Đưa bạn một bản nhạc in đầy nốt, bạn khó biết bài đó vui hay buồn. Bật bài hát lên, 3 giây là biết. Mắt người nhận ra **hình dạng, xu hướng, điểm lạc** nhanh hơn đọc số hàng nghìn lần. Biểu đồ là cách "bật bài hát" của

dữ liệu.

4.2. Matplotlib là "bộ bút chì màu", Seaborn là "bộ khuôn vẽ"

- **Matplotlib:** bộ bút chì đầy đủ. Vẽ gì cũng được, chỉnh từng nét được, nhưng phải tự tô từng chi tiết.
- **Seaborn:** bộ khuôn có sẵn, xây trên Matplotlib. Đặt khuôn vào, tô một lần là ra hình đẹp, có sẵn màu, chú thích, đường trung bình. Muốn chỉnh chi tiết thì lại cầm bút chì Matplotlib.

Người làm ML dùng **cả hai**: Seaborn để khám phá nhanh, Matplotlib để tinh chỉnh hình đưa vào báo cáo.

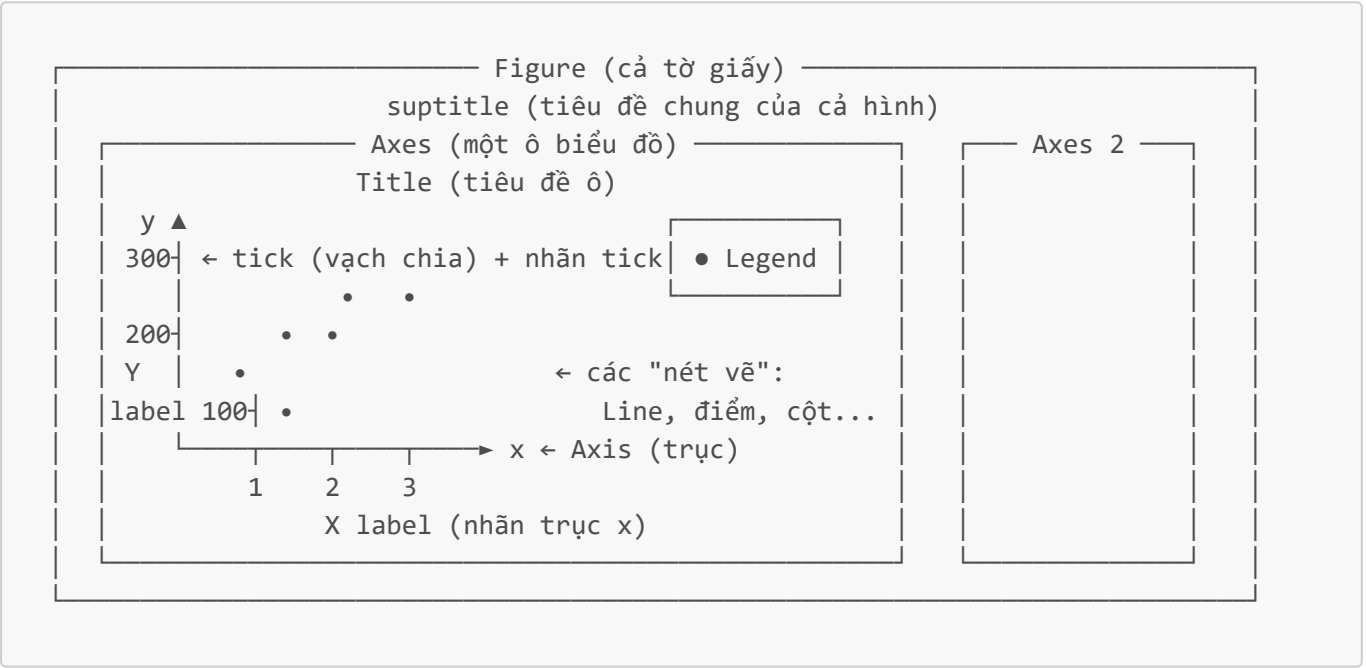
4.3. Mỗi câu hỏi có một loại biểu đồ

Bạn không dùng tua vít để đóng đinh. Hỏi "phân bố thế nào?" thì dùng histogram, hỏi "hai thứ có liên quan không?" thì dùng scatter. Chọn sai loại biểu đồ, người xem sẽ hiểu sai, dù số liệu đúng.

---

5. 📖 Lý thuyết chi tiết

5.1. Cấu tạo một biểu đồ Matplotlib



Thành phần	Là gì	Lệnh
Figure	Cả tờ giấy, chứa một hay nhiều ô	`fig = plt.figure(figsize=(8, 4))`
Axes	Một ô biểu đồ (chú ý: không phải "trục")	`ax = fig.add_subplot()` hoặc `plt.subplots()`
Axis	Trục x hoặc trục y của một ô	`ax.xaxis`, `ax.set_xlim(...)`
Tick	Vạch chia trên trục kèm nhãn	`ax.set_xticks(...)`
Title / Label	Tiêu đề ô / nhãn trục	`ax.set_title`, `ax.set_xlabel`

Thành phần	Là gì	Lệnh
Legend	Chú thích màu nào là gì	`ax.legend()`

5.2. Hai cách viết Matplotlib

```
# Cách 1: kiểu pyplot (plt.xxx) – nhanh, tiện thử nghiệm, nhưng khó kiểm soát khi có nhiều ô
plt.plot([1, 2, 3], [2, 4, 3])
plt.title("Cách 1")
plt.show()

# Cách 2: kiểu hướng đối tượng (fig, ax) – rõ ràng, bài này dùng chủ yếu cách này
fig, ax = plt.subplots()          # tạo tờ giấy (fig) và 1 ô (ax)
ax.plot([1, 2, 3], [2, 4, 3])     # vẽ LÊN Ô ax
ax.set_title("Cách 2")            # chú ý: plt.title ↔ ax.set_title
plt.show()
```

pyplot	Hướng đối tượng
`plt.title("...")`	`ax.set_title("...")`
`plt.xlabel("...")`	`ax.set_xlabel("...")`
`plt.xlim(0, 10)`	`ax.set_xlim(0, 10)`
`plt.legend()`	`ax.legend()`

Vì sao nên quen cách 2? Khi có 4 ô biểu đồ, bạn cần nói rõ "vẽ vào ô nào": `axes[0].plot(...)`, `axes[1].hist(...)`. Kiểu `plt.xxx` chỉ vẽ vào "ô hiện tại", rất dễ vẽ nhầm ô.

Mẫu code bạn sẽ dùng mãi:

```
fig, ax = plt.subplots(figsize=(8, 4))   # rộng 8, cao 4 (đơn vị inch)
ax.plot(...) / ax.scatter(...) / ax.bar(...) / ax.hist(...)
ax.set_xlabel("...") ; ax.set_ylabel("...") ; ax.set_title("...")
plt.show()
```

5.3. Chọn biểu đồ nào?

Câu hỏi bạn muốn trả lời	Biểu đồ	Matplotlib	Seaborn
Một biến số phân bố thế nào? (đa số nằm đâu, có lệch không)	**Histogram**, KDE	`ax.hist`	`sns.histplot`, `sns.kdeplot`
So sánh phân bố giữa các nhóm, tìm outlier	**Boxplot**, Violin	`ax.boxplot`	`sns.boxplot`, `sns.violinplot`

Câu hỏi bạn muốn trả lời	Biểu đồ	Matplotlib	Seaborn
Mỗi nhóm có bao nhiêu phần tử?	**Bar** (đếm)	`ax.bar`	`sns.countplot`
So sánh một con số (trung bình, tổng) giữa các nhóm	**Bar**	`ax.bar`, `ax.barh`	`sns.barplot`
Thay đổi theo thời gian	**Line**	`ax.plot`	`sns.lineplot`
Hai biến số có liên quan nhau không?	**Scatter**	`ax.scatter`	`sns.scatterplot`, `sns.regplot`
Nhiều cặp biến liên quan nhau thế nào?	**Heatmap**, Pairplot	`ax.imshow`	`sns.heatmap`, `sns.pairplot`
Tỉ lệ các phần của một tổng (2–4 phần)	Pie (hạn chế dùng)	`ax.pie`	(không có)

5.4. Màu sắc có ý nghĩa

Loại bảng màu (colormap)	Dùng khi	Ví dụ tên
Tuần tự (sequential)	Giá trị từ thấp đến cao	`viridis`, `Blues`, `magma`
Phân kỳ (diverging)	Có điểm giữa ý nghĩa (0, trung bình), hai phía đối lập	`coolwarm`, `RdBu`
Định tính (qualitative)	Các nhóm không có thứ tự	`tab10`, `Set2`

Quy tắc: **dữ liệu tương quan** (−1 đến 1) dùng màu phân kỳ; **giá nhà** (thấp → cao) dùng màu tuần tự; **loại nhà, giới tính** dùng màu định tính. Tránh bảng màu cầu vồng `jet`: mắt người thấy vùng màu vàng "nổi bật" giả tạo, và người mù màu không phân biệt được.

5.5. Seaborn: hai nhóm hàm

Nhóm	Ví dụ	Vẽ vào	Dùng khi
Axes-level (vẽ một ô)	`histplot`, `boxplot`, `scatterplot`, `barplot`, `heatmap`	Một `ax` (truyền `ax=...` được)	Ghép vào lưới subplot của bạn
Figure-level (tự tạo cả hình)	`displot`, `catplot`, `relplot`, `pairplot`, `jointplot`	Cả một Figure mới	Muốn chia nhiều ô theo nhóm bằng `col=`, `row=`

Mọi hàm Seaborn có cùng "ngôn ngữ":

- `data=df`: bảng dữ liệu (dạng dài).
- `x="cột"`, `y="cột"`: tên cột cho mỗi trục.
- `hue="cột"`: tô màu theo nhóm.
- `col="cột"` / `row="cột"` (figure-level): tách thành nhiều ô theo nhóm.

5.6. 📌 Thuật ngữ

Tên tiếng Anh	Nghĩa tiếng Việt	Giải thích 1 câu	Ví dụ
Data visualization	Trực quan hóa dữ liệu	Biến số liệu thành hình để mắt dễ hiểu.	Biểu đồ doanh thu
Figure / Axes	Hình / Ô biểu đồ	Tờ giấy / một biểu đồ trên tờ giấy.	`fig, ax = plt.subplots()`
Histogram	Biểu đồ tần suất	Chia giá trị thành các thùng, đếm số điểm mỗi thùng.	Phổ điểm thi
Bin	Thùng	Một khoảng giá trị trong histogram.	6,0–6,5 điểm
KDE (Kernel Density Estimate)	Ước lượng mật độ	Đường cong mượt thay cho các cột histogram.	Hình "quả chuông"
Scatter plot	Biểu đồ điểm	Mỗi mẫu là một chấm theo 2 giá trị.	Thu nhập vs giá nhà
Boxplot	Biểu đồ hộp	Tóm tắt phân bố bằng trung vị, tứ phân vị, outlier.	Lương theo phòng ban
Violin plot	Biểu đồ đàn violin	Boxplot kết hợp hình dạng phân bố.	Tiền boa theo ngày
Heatmap	Bản đồ nhiệt	Bảng số được tô màu theo độ lớn.	Bảng tương quan
Colormap	Bảng màu	Quy tắc đổi số thành màu.	`viridis`
Legend	Chú thích	Hộp giải thích màu/ký hiệu.	● Cũ ● Mới
Overplotting	Chồng điểm	Quá nhiều điểm đè lên nhau, không thấy mật độ.	17.000 chấm đặc kín
Facet	Chia ô theo nhóm	Vẽ cùng loại biểu đồ cho từng nhóm, đặt cạnh nhau.	`col="loai_nha"`
Error bar / CI	Thanh sai số / khoảng tin cậy	Vạch cho biết con số trung bình "chắc chắn" đến đâu.	Vạch đen trên cột barplot

6. 💻 Code từng bước (Colab)

> Chạy các ô **theo đúng thứ tự**. Biểu đồ hiện ngay dưới ô code. Phần "Bạn sẽ thấy" mô tả hình bằng ký tự để bạn đối chiếu.

6.1. Chuẩn bị

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns          # quy ước tên tắt là sns
```

```
url = "https://download.mlcc.google.com/mledu-
datasets/california_housing_train.csv"
df = pd.read_csv(url)
df.columns = ["kinh_do", "vi_do", "tuoi_nha", "tong_phong", "tong_phong_ngu",
              "dan_so", "so_ho", "thu_nhap", "gia_nha"]           # đổi tên cột
(cách nhanh)
df["loai_nha"] = np.where(df["tuoi_nha"] >= 30, "Cũ", "Mới")      # như bài 1.4
df["phong_moi_ho"] = df["tong_phong"] / df["so_ho"]
print(df.shape)
```

Kết quả mẫu:

```
(17000, 11)
```

6.2. Line chart (Biểu đồ đường)

6.2.1. Đường đơn giản

Dùng cho dữ liệu **có thứ tự**, thường là **thời gian**. Ví dụ doanh thu một quán cà phê:

```
thang = ["T1", "T2", "T3", "T4", "T5", "T6", "T7", "T8", "T9", "T10", "T11",
"T12"]
cn_a = [120, 95, 140, 160, 155, 190, 210, 205, 170, 165, 180, 240]   # chi nhánh A
(triệu đồng)
cn_b = [100, 110, 115, 120, 130, 125, 140, 150, 160, 175, 185, 200]  # chi nhánh B

fig, ax = plt.subplots(figsize=(8, 3))
ax.plot(thang, cn_a, marker="o")      # marker="o": đánh dấu từng điểm bằng chấm
tròn
ax.set_ylabel("Doanh thu (triệu đồng)")
ax.set_title("Doanh thu chi nhánh A năm 2025")
plt.show()
```

Bạn sẽ thấy: một đường gấp khúc đi lên, giảm nhẹ ở T2 (Tết), đỉnh nhỏ ở T7 (hè), vọt lên ở T12 (cuối năm).

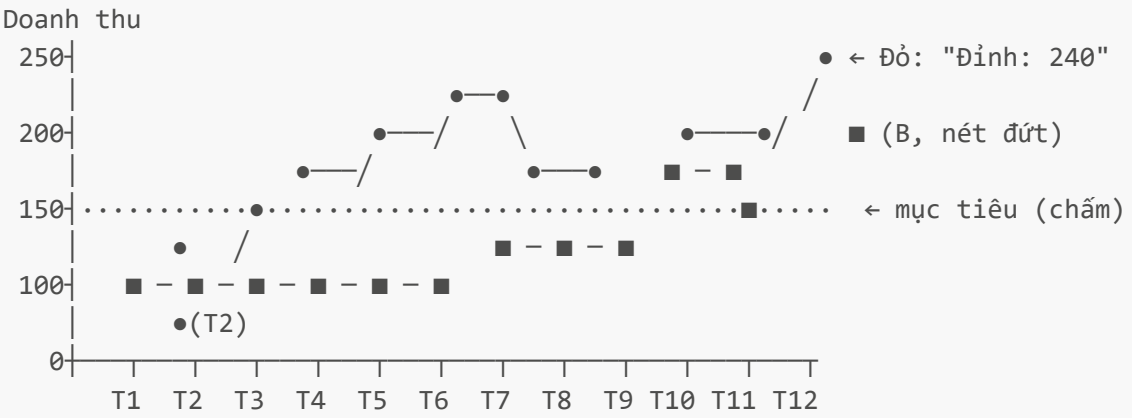
6.2.2. Nhiều đường, chú thích, tùy chỉnh nét vẽ

```
fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(thang, cn_a, color="tab:blue", marker="o", linewidth=2, label="Chi nhánh
A")
ax.plot(thang, cn_b, color="tab:orange", marker="s", linestyle="--", label="Chi
nhánh B")  # s = square, -- = nét đứt
ax.axhline(150, color="gray", linestyle=":", label="Mục tiêu 150")   # đường ngang
tham chiếu
```

```
# Chú thích điểm cao nhất bằng mũi tên
i_max = int(np.argmax(cn_a))
ax.annotate(f"Đỉnh: {cn_a[i_max]}", xy=(i_max, cn_a[i_max]),     # điểm được chỉ
vào
            xytext=(i_max - 3, cn_a[i_max] + 5),                 # vị trí đặt chữ
            arrowprops=dict(arrowstyle="->"))

ax.set_ylim(0, 260)                    # trục y bắt đầu từ 0 để không phóng đại
chênh lệch
ax.set_xlabel("Tháng")
ax.set_ylabel("Doanh thu (triệu đồng)")
ax.set_title("Chi nhánh B tăng đều, chi nhánh A biến động mạnh")
ax.legend(loc="upper left")            # hiện chú thích từ các label=
ax.grid(alpha=0.3)                     # lưới mờ cho dễ đọc
plt.show()
```

Bạn sẽ thấy:



Tham số	Ý nghĩa	Giá trị hay dùng
`color`	Màu	`"tab:blue"`, `"red"`, `"#1f77b4"`
`linestyle`	Kiểu nét	`"-"` liền, `"--"` đứt, `":"` chấm, `"-."`
`marker`	Ký hiệu điểm	`"o"` tròn, `"s"` vuông, `"^"` tam giác, `"x"`
`linewidth`	Độ dày nét	`1`, `2`, `3`
`label`	Tên hiện trong legend	`"Chi nhánh A"`
`alpha`	Độ trong suốt	`0` (vô hình) → `1` (đặc)

Bạn sẽ dùng biểu đồ đường rất nhiều ở Giai đoạn 3 để vẽ **loss qua từng vòng huấn luyện** (sẽ học kỹ ở bài 3.5).

6.2.3. Vẽ một hàm toán học

```
x = np.linspace(-5, 5, 200)                  # 200 điểm cách đều để đường cong
mượt (bài 1.3)
fig, ax = plt.subplots(figsize=(7, 3))
ax.plot(x, 1 / (1 + np.exp(-x)), label="sigmoid")     # hàm sigmoid (sẽ gặp ở bài
2.4)
ax.plot(x, np.maximum(0, x) / 5, label="ReLU / 5")    # hàm ReLU thu nhỏ (bài 1.3,
3.3)
ax.axvline(0, color="black", linewidth=0.5)           # trục dọc tại x = 0
ax.legend()
ax.set_title("Hai hàm kích hoạt nổi tiếng")
plt.show()
```

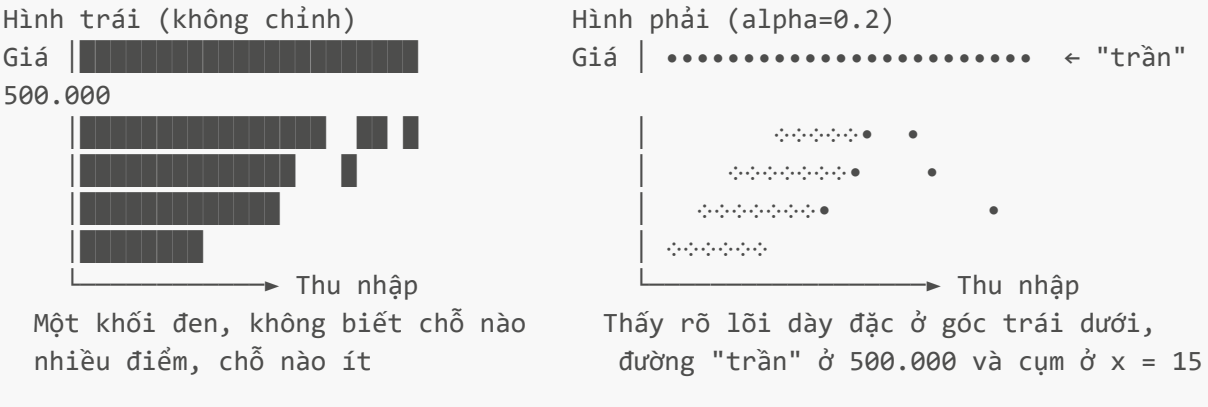
Bạn sẽ thấy: đường **sigmoid** hình chữ S, nằm giữa 0 và 1, đi qua 0,5 tại x = 0. Đường **ReLU** nằm sát 0 bên trái, rồi thẳng đi lên bên phải.

6.3. Scatter plot (Biểu đồ điểm)

6.3.1. Cơ bản và vấn đề "chồng điểm"

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].scatter(df["thu_nhap"], df["gia_nha"])                    # mặc định:
chấm to, đặc
axes[0].set_title("Không chỉnh: 17.000 chấm đè lên nhau")
axes[1].scatter(df["thu_nhap"], df["gia_nha"], s=3, alpha=0.2)    # s: kích
thước; alpha: độ trong suốt
axes[1].set_title("s=3, alpha=0.2: thấy được chỗ dày đặc")
for ax in axes:
    ax.set_xlabel("Thu nhập (chục nghìn USD)")
    ax.set_ylabel("Giá nhà (USD)")
plt.tight_layout()
plt.show()
```

Bạn sẽ thấy:



Đọc biểu đồ bên phải:

1. Đám mây điểm đi lên từ trái sang phải → thu nhập và giá nhà **cùng tăng**. Đây là đặc trưng quan trọng.
2. Một đường ngang dày ở 500.000 (**giới hạn trần**, bài 1.4). Nhìn kỹ còn có vài đường ngang mờ hơn ở 112.500, 137.500, 162.500, 187.500...: các giá "tròn" mà người khai đã làm tròn (bài 1.4, mục 6.14). Đây là loại chi tiết chỉ biểu đồ mới cho thấy.
3. Một cột điểm ở thu nhập = 15 (giới hạn trần của thu nhập).

6.3.2. Thêm chiều thứ ba bằng màu và kích thước

```
mau = df.sample(2000, random_state=0)        # lấy mẫu 2.000 dòng cho hình thoáng hơn

fig, ax = plt.subplots(figsize=(8, 5))
diem = ax.scatter(mau["thu_nhap"], mau["gia_nha"],
                  c=mau["tuoi_nha"],         # c: MÀU theo tuổi nhà (cột số → dùng colormap)
                  cmap="viridis",            # tím (mới) → vàng (cũ)
                  s=mau["dan_so"] / 100,     # kích thước theo dân số
                  alpha=0.6, edgecolor="none")
fig.colorbar(diem, ax=ax, label="Tuổi nhà (năm)")   # thanh chú thích màu
ax.set_xlabel("Thu nhập (chục nghìn USD)")
ax.set_ylabel("Giá nhà (USD)")
ax.set_title("Thu nhập, giá, tuổi nhà và dân số trên cùng một hình")
plt.show()
```

Bạn sẽ thấy: đám mây đi lên như trước, các màu từ tím (nhà mới) đến vàng (nhà cũ) **trộn lẫn khắp nơi**, không tách thành vùng rõ ràng. Đó cũng là một phát hiện: tuổi nhà không quyết định giá mạnh như thu nhập. Một biểu đồ, bốn thông tin: thu nhập, giá, tuổi nhà, dân số. Nhưng nhiều thông tin quá thì mắt khó đọc, nên ở mục 6.15 ta sẽ dùng heatmap để tách bạch tác động của thu nhập và tuổi nhà.

6.4. Bar chart (Biểu đồ cột)

6.4.1. Cột đứng từ kết quả groupby

```
# Giá trung bình theo nhóm tuổi nhà. pd.cut chia số thành các khoảng (bài 1.4)
nhom_tuoi = pd.cut(df["tuoi_nha"], bins=[0, 10, 20, 30, 40, 52],
                   labels=["0-10", "11-20", "21-30", "31-40", "41-52"])
gia_tb = df.groupby(nhom_tuoi, observed=True)["gia_nha"].mean()
print(gia_tb.round(0))

fig, ax = plt.subplots(figsize=(7, 4))
cot = ax.bar(gia_tb.index.astype(str), gia_tb.values, color="coral")   # trục x là chữ
ax.bar_label(cot, fmt="%.0f", fontsize=8)     # ghi con số lên đầu mỗi cột
ax.set_xlabel("Tuổi nhà (năm)")
ax.set_ylabel("Giá trung bình (USD)")
ax.set_title("Nhà trên 40 năm tuổi có giá trung bình cao nhất")
plt.show()
```

Kết quả mẫu:

```
tuoi_nha
0-10     200928.0
11-20    191312.0
21-30    207208.0
31-40    206814.0
41-52    230420.0
Name: gia_nha, dtype: float64
```

6.4.2. Cột ngang, đã sắp xếp

Khi tên nhóm dài hoặc có nhiều nhóm, dùng cột **ngang** và **sắp xếp** để người xem so sánh ngay:

```
ban_chay = pd.Series({"Cà phê sữa": 420, "Trà đào cam sả": 310, "Bạc xỉu": 280,
                      "Trà sữa trân châu": 450, "Nước ép cam": 120, "Sinh tố bơ":
95})
ban_chay = ban_chay.sort_values()                       # tăng dần → cột dài nhất
nằm TRÊN CÙNG

fig, ax = plt.subplots(figsize=(7, 4))
ax.barh(ban_chay.index, ban_chay.values, color="tab:green")   # barh = bar
horizontal
ax.set_xlabel("Số ly bán trong tháng")
ax.set_title("Trà sữa trân châu bán chạy nhất")
plt.show()
```

Bạn sẽ thấy:

```
Trà sữa trân châu ████████████████████ 450
       Cà phê sữa ██████████████████ 420
   Trà đào cam sả ██████████████ 310
          Bạc xỉu ██████████████ 280
      Nước ép cam ██████ 120
       Sinh tố bơ ██████ 95
```

6.4.3. Cột nhóm (grouped) và cột chồng (stacked)

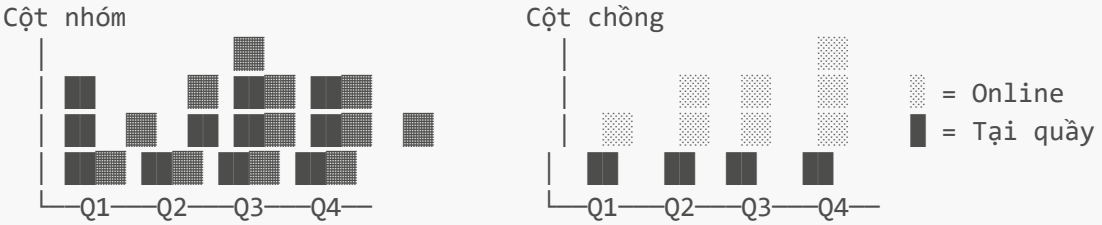
```
quy = ["Q1", "Q2", "Q3", "Q4"]
online = np.array([30, 45, 50, 70])          # doanh thu online (tỷ)
tai_quay = np.array([60, 55, 52, 50])        # doanh thu tại quầy
vi_tri = np.arange(len(quy))                 # vị trí 0, 1, 2, 3 trên trục x
rong = 0.4                                   # độ rộng mỗi cột

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
```

```
# Cột nhóm: dịch 2 cột sang trái/phải một nửa độ rộng để đứng cạnh nhau
axes[0].bar(vi_tri - rong / 2, online, rong, label="Online")
axes[0].bar(vi_tri + rong / 2, tai_quay, rong, label="Tại quầy")
axes[0].set_xticks(vi_tri, quy)              # đặt lại nhãn trục x
axes[0].set_title("So sánh từng kênh: online vượt tại quầy ở Q4")
axes[0].legend()

# Cột chồng: cột thứ hai bắt đầu từ đỉnh cột thứ nhất (bottom=)
axes[1].bar(quy, tai_quay, label="Tại quầy")
axes[1].bar(quy, online, bottom=tai_quay, label="Online")
axes[1].set_title("Tổng doanh thu tăng, phần online ngày càng lớn")
axes[1].legend()
plt.tight_layout()
plt.show()
```

Bạn sẽ thấy:



Dùng	Khi
Cột nhóm	Muốn so sánh **từng thành phần** giữa các nhóm
Cột chồng	Muốn thấy **tổng** và **cơ cấu** của tổng

6.5. Histogram (Biểu đồ tần suất)

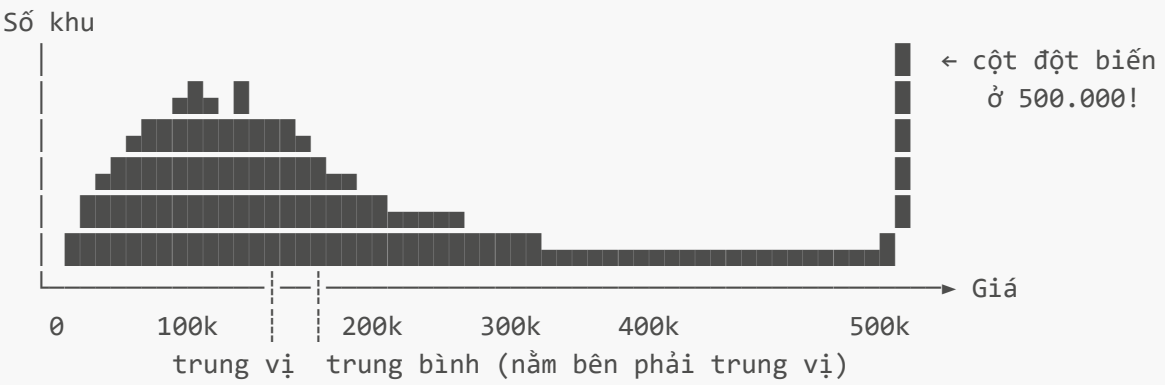
6.5.1. Cơ bản

- **Định nghĩa:** Chia khoảng giá trị thành nhiều "thùng" (bin), rồi đếm có bao nhiêu điểm rơi vào mỗi thùng.
- **Ví dụ đời thực:** Phổ điểm thi đại học: bao nhiêu thí sinh được 5 điểm, 6 điểm, 7 điểm...

```
fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(df["gia_nha"], bins=50, color="steelblue", edgecolor="white")  # chia 50
thùng
ax.axvline(df["gia_nha"].mean(), color="red", linestyle="--", label="Trung bình")
ax.axvline(df["gia_nha"].median(), color="black", linestyle=":", label="Trung vị")
ax.set_xlabel("Giá nhà (USD)")
ax.set_ylabel("Số khu dân cư")
ax.set_title("Giá nhà lệch phải và bị giới hạn trần ở 500.000")
```

```
ax.legend()
plt.show()
```

Bạn sẽ thấy:



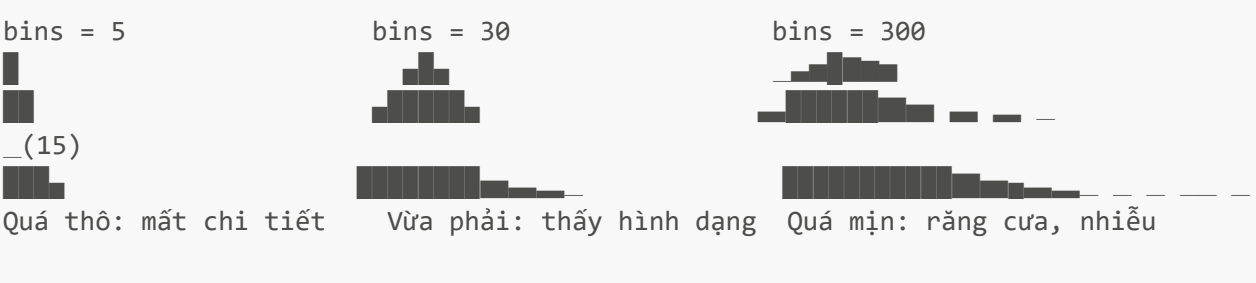
Đọc biểu đồ:

1. Đa số khu có giá 100.000–250.000 USD.
2. Phân bố **lệch phải**: đuôi kéo dài về phía giá cao. Vì vậy **trung bình > trung vị** (đường đỏ nằm bên phải đường đen) (sẽ học kỹ ở bài 1.7).
3. Cột cao vọt ở 500.000: chính là **giới hạn trần** bạn đã phát hiện ở bài 1.4. Biểu đồ cho thấy nó ngay lập tức, không cần đếm.

6.5.2. Số thùng ảnh hưởng thế nào?

```
fig, axes = plt.subplots(1, 3, figsize=(14, 3))
for ax, b in zip(axes, [5, 30, 300]):
    ax.hist(df["thu_nhap"], bins=b)
    ax.set_title(f"bins = {b}")
plt.tight_layout()
plt.show()
```

Bạn sẽ thấy:



Không có số thùng "đúng". Thử vài giá trị, chọn giá trị **thấy rõ hình dạng nhất**. Mặc định là 10 thùng, thường quá ít với dữ liệu lớn.

6.5.3. So sánh hai nhóm trên cùng một hình

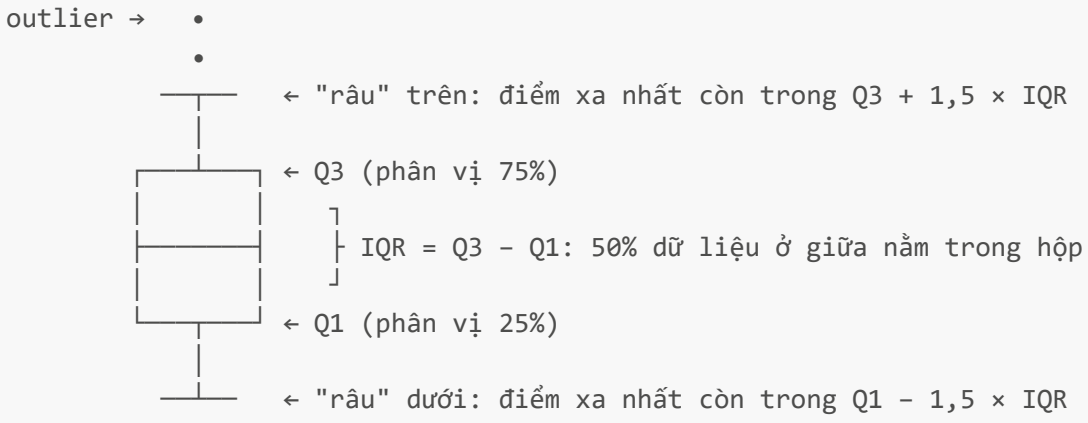
```
fig, ax = plt.subplots(figsize=(8, 4))
for loai, mau_ve in [("Cũ", "tab:red"), ("Mới", "tab:blue")]:
    ax.hist(df.loc[df["loai_nha"] == loai, "thu_nhap"], bins=40,
            alpha=0.5, color=mau_ve, label=loai, density=True)   # density=True:
chuẩn hóa để so sánh nhóm khác cỡ
ax.set_xlabel("Thu nhập (chục nghìn USD)")
ax.set_ylabel("Mật độ")
ax.legend()
ax.set_title("Khu nhà mới có thu nhập nhỉnh hơn")
plt.show()
```

Bạn sẽ thấy: hai "ngọn núi" chồng lên nhau, phần trong suốt cho thấy chỗ giao. Núi màu xanh (Mới) lệch sang phải một chút.

`density=True` đổi chiều cao cột từ "số lượng" sang "mật độ", sao cho **tổng diện tích các cột = 1**. Nhờ vậy, hai nhóm có số lượng khác nhau vẫn so sánh hình dạng công bằng.

6.6. Boxplot (Biểu đồ hộp)

- **Định nghĩa:** Tóm tắt phân bố bằng 5 con số: nhỏ nhất, phân vị 25% (Q1), trung vị, phân vị 75% (Q3), lớn nhất. Điểm nằm quá xa được vẽ riêng là outlier.
- **Ví dụ đời thực:** So sánh lương của nhân viên giữa các phòng ban chỉ bằng một hình.



Tính tay với 9 điểm thi: `[3, 5, 6, 6, 7, 7, 8, 9, 15]`

- Trung vị = 7 (điểm thứ 5). Q1 = 6, Q3 = 8 → IQR = 8 − 6 = 2.
- Râu trên tối đa: 8 + 1,5 × 2 = 11. Râu dưới tối thiểu: 6 − 1,5 × 2 = 3.
- Điểm **15 > 11** → là outlier, vẽ thành chấm riêng. Điểm 3 vừa chạm biên dưới → vẫn nằm trong râu.

```
diem = [3, 5, 6, 6, 7, 7, 8, 9, 15]
print(np.percentile(diem, [25, 50, 75]))      # kiểm tra Q1, trung vị, Q3

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
```

```
axes[0].boxplot(diem)                          # boxplot của Matplotlib
axes[0].set_title("9 điểm thi: điểm 15 là outlier")
nhom = [df.loc[df["loai_nha"] == l, "gia_nha"] for l in ["Cũ", "Mới"]]
axes[1].boxplot(nhom)
axes[1].set_xticks([1, 2], ["Cũ", "Mới"])      # boxplot đặt các hộp ở vị trí 1,
2, ...
axes[1].set_title("Giá nhà theo loại nhà")
plt.show()
```

Kết quả mẫu:

```
[6. 7. 8.]
```

Bạn sẽ thấy: hình trái có một hộp hẹp, điểm 15 nằm riêng phía trên. Hình phải có hai hộp gần giống nhau, đường trung vị của nhà Cũ cao hơn một chút; cả hai có nhiều chấm outlier sát mức 500.000.

Seaborn vẽ boxplot đẹp và tiện hơn (mục 6.13). Bài 1.6 sẽ dùng quy tắc 1,5 × IQR này để **phát hiện outlier** bằng code.

6.7. Pie chart (Biểu đồ tròn): dùng rất hạn chế

```
kenh = pd.Series({"GrabFood": 45, "ShopeeFood": 30, "Tại quán": 20, "Khác": 5})
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].pie(kenh.values, labels=kenh.index, autopct="%1.0f%%", startangle=90)   #
autopct: ghi % lên lát
axes[0].set_title("Pie: khó so ShopeeFood với Tại quán")
axes[1].barh(kenh.index[::-1], kenh.values[::-1])
axes[1].set_title("Bar: so sánh ngay lập tức")
plt.show()
```

Bạn sẽ thấy: biểu đồ tròn có 4 lát; biểu đồ cột ngang bên cạnh. Mắt người so sánh **độ dài** chính xác hơn nhiều so với **góc** và **diện tích**.

Quy tắc: chỉ dùng pie khi có **2–4 phần** và muốn nhấn mạnh "một phần của tổng". Không bao giờ dùng pie 3D.

6.8. Nhiều biểu đồ trên một hình: `subplots`

```
fig, axes = plt.subplots(2, 2, figsize=(11, 8))   # lưới 2 hàng × 2 cột
print(axes.shape)                                  # axes là mảng NumPy 2 chiều
chứa 4 ô

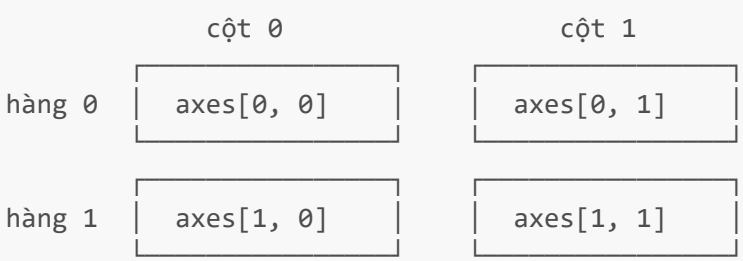
axes[0, 0].hist(df["tuoi_nha"], bins=52, color="gray")
axes[0, 0].set_title("Tuổi nhà")
axes[0, 1].hist(df["thu_nhap"], bins=50, color="tab:green")
axes[0, 1].set_title("Thu nhập")
axes[1, 0].scatter(df["tuoi_nha"], df["gia_nha"], s=2, alpha=0.1)
```

```
axes[1, 0].set_title("Tuổi nhà vs giá")
axes[1, 1].scatter(df["thu_nhap"], df["gia_nha"], s=2, alpha=0.1,
color="tab:green")
axes[1, 1].set_title("Thu nhập vs giá")

fig.suptitle("Tổng quan dữ liệu nhà California", fontsize=14)   # tiêu đề chung
plt.tight_layout()
plt.show()
```

Kết quả mẫu:

```
(2, 2)
```



Bạn sẽ thấy: ô "Tuổi nhà" có một cột cao vọt ở 52 (giới hạn trần thứ ba!). Ô "Tuổi nhà vs giá" là một đám mây gần như phẳng, ô "Thu nhập vs giá" đi lên rõ rệt: thu nhập liên quan đến giá mạnh hơn tuổi nhà nhiều.

Mẹo duyệt mọi ô bằng vòng lặp: `for ax in axes.flat:` (`.flat` duỗi phẳng mảng ô, bài 1.3). Thêm `sharey=True` vào `plt.subplots(...)` để các ô dùng chung thang trục y, so sánh công bằng hơn.

6.9. Tùy chỉnh để biểu đồ chuyên nghiệp

6.9.1. Định dạng số trên trục

```
import matplotlib.ticker as mtick      # công cụ định dạng vạch chia

fig, ax = plt.subplots(figsize=(7, 3))
ax.hist(df["gia_nha"], bins=40)
ax.xaxis.set_major_formatter(mtick.StrMethodFormatter("{x:,.0f}"))   # 500000 →
500,000
ax.tick_params(axis="x", rotation=30)                                # xoay nhãn
30° cho khỏi đè nhau
ax.set_xlabel("Giá nhà (USD)")
plt.show()
```

Bạn sẽ thấy: nhãn trục x hiện `100,000`, `200,000`... thay vì `1e5` hay `100000` khó đọc, và được xoay nghiêng.

6.9.2. Phong cách và cỡ chữ

```
print(plt.style.available[:6])              # một vài phong cách có sẵn
plt.rcParams["font.size"] = 11              # đổi cỡ chữ mặc định cho MỌI biểu đồ
sau đó

with plt.style.context("ggplot"):           # dùng phong cách ggplot CHỈ trong khối
with này
    fig, ax = plt.subplots(figsize=(6, 3))
    ax.plot(thang, cn_b, marker="o")
    ax.set_title("Phong cách ggplot")
    plt.show()
```

Kết quả mẫu:

```
['Solarize_Light2', 'bmh', 'classic', 'dark_background', 'fast',
'fivethirtyeight']
```

`rcParams` là "cài đặt chung" của Matplotlib. Hay chỉnh: `figure.figsize`, `font.size`, `axes.grid`.

6.10. Lưu biểu đồ thành file ảnh

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.scatter(df["thu_nhap"], df["gia_nha"], s=2, alpha=0.2)
ax.set_title("Thu nhập càng cao, giá nhà càng cao")
fig.savefig("thu_nhap_gia.png", dpi=150, bbox_inches="tight")   # dpi: độ nét;
tight: cắt viền thừa
plt.close(fig)                                                   # đóng hình,
không hiển thị
import os; print(os.path.exists("thu_nhap_gia.png"))
```

Kết quả mẫu:

```
True
```

- Gọi `savefig` **trước** `plt.show()`. Sau `show`, hình đã được "xả" và file lưu ra có thể trống.
- `.png` cho báo cáo, slide, Notion. `.svg` / `.pdf` cho hình vector (phóng to không vỡ).

6.11. Seaborn: làm quen

```
sns.set_theme(style="whitegrid")          # giao diện nền trắng có lưới cho mọi
biểu đồ sau đó
tips = sns.load_dataset("tips")           # dataset có sẵn của Seaborn (cần
internet)
```

```
print(tips.shape)
print(tips.head())
```

Kết quả mẫu:

```
(244, 7)
   total_bill   tip     sex smoker  day    time  size
0       16.99  1.01  Female     No  Sun  Dinner     2
1       10.34  1.66    Male     No  Sun  Dinner     3
2       21.01  3.50    Male     No  Sun  Dinner     3
3       23.68  3.31    Male     No  Sun  Dinner     2
4       24.59  3.61  Female     No  Sun  Dinner     4
```

Dataset tips (tiền boa): 244 hóa đơn ở một nhà hàng tại Mỹ, do một người phục vụ ghi lại. Cột: `total_bill` (tổng hóa đơn, USD), `tip` (tiền boa), `sex` (giới tính người trả), `smoker` (có hút thuốc), `day` (thứ), `time` (bữa trưa/tối), `size` (số người trong bàn).

Cú pháp chung của Seaborn, so với Matplotlib:

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].scatter(tips["total_bill"], tips["tip"])                              #
Matplotlib: truyền từng cột
axes[0].set_xlabel("total_bill"); axes[0].set_ylabel("tip")
sns.scatterplot(data=tips, x="total_bill", y="tip", hue="time", ax=axes[1])   #
Seaborn: tên cột + hue
plt.show()
```

Bạn sẽ thấy: cùng một đám mây điểm. Bên Seaborn có sẵn nhãn trục, màu theo bữa ăn (Lunch/Dinner) và legend, chỉ với một dòng.

6.12. Seaborn: biểu đồ phân bố

6.12.1. `histplot`

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

sns.histplot(data=df, x="thu_nhap", bins=40, kde=True, ax=axes[0])     # kde=True:
thêm đường cong mượt
axes[0].set_title("Thu nhập + đường KDE")

sns.histplot(data=df, x="thu_nhap", hue="loai_nha", bins=40,
             stat="density", common_norm=False, element="step", ax=axes[1])   #
mỗi nhóm chuẩn hóa riêng, vẽ viền
axes[1].set_title("So sánh 2 nhóm (density)")

sns.histplot(data=df, x="thu_nhap", hue="loai_nha", bins=40,
```

```
                multiple="stack", ax=axes[2])                                  # chồng 2
nhóm lên nhau
axes[2].set_title("Chồng 2 nhóm (stack)")
plt.tight_layout()
plt.show()
```

Bạn sẽ thấy: hình 1 là histogram có đường cong mượt bám theo đỉnh cột; hình 2 là hai đường bậc thang cho Cũ và Mới, dễ so hình dạng; hình 3 là các cột hai màu chồng lên nhau, tổng chiều cao bằng histogram chung.

Tham số	Ý nghĩa
`bins`	Số thùng
`kde=True`	Thêm đường mật độ mượt
`hue`	Tách màu theo nhóm
`stat`	`"count"` (mặc định), `"density"`, `"percent"`
`common_norm=False`	Mỗi nhóm chuẩn hóa riêng (so hình dạng công bằng)
`multiple`	`"layer"` (chồng trong suốt), `"stack"`, `"dodge"`, `"fill"`
`element`	`"bars"`, `"step"` (viền bậc thang)

6.12.2. `kdeplot`: đường mật độ

```
fig, ax = plt.subplots(figsize=(8, 4))
sns.kdeplot(data=tips, x="tip", hue="time", fill=True, common_norm=False, ax=ax)
ax.set_title("Bữa tối thường boa nhiều hơn bữa trưa một chút")
plt.show()
```

Bạn sẽ thấy: hai "ngọn đồi" mượt được tô màu. Đồi Dinner lệch sang phải và có đuôi dài hơn.

> 💡 KDE làm mượt dữ liệu nên có thể "vẽ lan" sang vùng không có dữ liệu thật (ví dụ tiền boa âm). Với dữ liệu ít điểm, histogram trung thực hơn.

6.13. Seaborn: so sánh các nhóm

6.13.1. `countplot`: đếm số lượng mỗi nhóm

```
fig, ax = plt.subplots(figsize=(6, 4))
sns.countplot(data=tips, x="day", hue="time", order=["Thur", "Fri", "Sat", "Sun"],
ax=ax)  # order: thứ tự cột
ax.set_title("Thứ Bảy, Chủ Nhật đông khách nhất; trưa chủ yếu là thứ Năm")
plt.show()
print(pd.crosstab(tips["day"], tips["time"]))    # kiểm tra bằng số (bài 1.4)
```

Kết quả mẫu:

```
time  Lunch  Dinner
day
Thur     61       1
Fri       7      12
Sat       0      87
Sun       0      76
```

6.13.2. `barplot`: trung bình mỗi nhóm kèm sai số

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.barplot(data=tips, x="day", y="tip", order=["Thur", "Fri", "Sat", "Sun"],
ax=axes[0])  # mặc định: TRUNG BÌNH
axes[0].set_title("Tiền boa trung bình theo ngày (vạch đen: khoảng tin cậy 95%)")

sns.barplot(data=tips, x="day", y="tip", order=["Thur", "Fri", "Sat", "Sun"],
            estimator="median", errorbar=None, ax=axes[1])    # dùng TRUNG VỊ, bỏ
vạch sai số
axes[1].set_title("Tiền boa trung vị theo ngày")
plt.show()
print(tips.groupby("day", observed=True)["tip"].agg(["mean", "median",
"count"]).round(2))
```

Kết quả mẫu:

```
      mean  median  count
day
Thur  2.77    2.30     62
Fri   2.73    3.00     19
Sat   2.99    2.75     87
Sun   3.26    3.15     76
```

Đọc vạch đen (errorbar): đó là **khoảng tin cậy 95%** của trung bình (sẽ học kỹ ở bài 1.10). Vạch **dài** nghĩa là ta **ít chắc chắn** về con số trung bình, thường vì nhóm đó có ít dữ liệu. Thứ Sáu chỉ có 19 hóa đơn nên vạch dài nhất. Hai cột có vạch đen **chồng lên nhau nhiều** thì chưa đủ bằng chứng để nói nhóm nào hơn nhóm nào.

> ⚠️ `barplot` **không** đếm số dòng. Nó tính **trung bình** cột `y`. Muốn đếm, dùng `countplot`.

6.13.3. `boxplot`, `violinplot`, `stripplot`

```
fig, axes = plt.subplots(1, 3, figsize=(16, 4))
thu_tu = ["Thur", "Fri", "Sat", "Sun"]

sns.boxplot(data=tips, x="day", y="total_bill", order=thu_tu, ax=axes[0])
axes[0].set_title("Boxplot")
```

```
sns.violinplot(data=tips, x="day", y="total_bill", order=thu_tu, inner="quartile",
ax=axes[1])  # inner: vẽ các tứ phân vị
axes[1].set_title("Violin: thấy cả hình dạng phân bố")

sns.boxplot(data=tips, x="day", y="total_bill", order=thu_tu, color="white",
ax=axes[2])
sns.stripplot(data=tips, x="day", y="total_bill", order=thu_tu, alpha=0.5,
ax=axes[2])   # vẽ TỪNG điểm lên trên
axes[2].set_title("Box + từng điểm thật")
plt.tight_layout()
plt.show()
```

Bạn sẽ thấy:



Hóa đơn cuối tuần (Sat, Sun) cao hơn và phân tán rộng hơn. Violin cho thấy phần lớn hóa đơn tập trung quanh 15–20 USD, với đuôi dài phía trên.

Biểu đồ	Ưu điểm	Nhược điểm
Boxplot	Gọn, thấy ngay trung vị và outlier	Giấu hình dạng (2 đỉnh vẫn trông như 1 hộp)
Violin	Thấy hình dạng phân bố	Khó đọc với người không quen
Strip/Swarm	Thấy từng điểm thật, thấy cỡ mẫu	Rối khi có hàng nghìn điểm

6.14. Seaborn: quan hệ giữa các biến

6.14.1. `scatterplot` với `hue`, `size`, `style`

```
fig, ax = plt.subplots(figsize=(8, 5))
sns.scatterplot(data=tips, x="total_bill", y="tip",
                hue="time",          # màu theo bữa
                size="size",         # kích thước chấm theo số người trong bàn
                style="smoker",      # hình dạng chấm theo hút thuốc / không
                alpha=0.7, ax=ax)
ax.set_title("Hóa đơn càng lớn, tiền boa càng nhiều")
plt.show()
```

Bạn sẽ thấy: đám mây đi lên; chấm to (bàn đông người) nằm phía phải (hóa đơn lớn). Seaborn tự tạo legend cho cả ba thông tin.

> ⚠️ Đừng nhồi quá 3–4 thông tin vào một biểu đồ. Người xem sẽ bị rối.

6.14.2. `regplot`: thêm đường xu hướng

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.regplot(data=tips, x="total_bill", y="tip", ax=axes[0],
            scatter_kws={"alpha": 0.4})            # truyền tùy chỉnh cho phần
chấm điểm
axes[0].set_title("Đường thẳng xu hướng + vùng tin cậy")

mau = df.sample(2000, random_state=1)
sns.regplot(data=mau, x="thu_nhap", y="gia_nha", ax=axes[1],
            scatter_kws={"s": 4, "alpha": 0.3}, line_kws={"color": "red"})
axes[1].set_title("Thu nhập vs giá nhà (2.000 khu)")
plt.show()

print(tips[["total_bill", "tip"]].corr().round(2))     # hệ số tương quan (bài
1.7)
```

Kết quả mẫu:

```
            total_bill   tip
total_bill        1.00  0.68
tip               0.68  1.00
```

Đường đỏ là **đường thẳng khớp dữ liệu nhất**. Đó chính là thứ **Linear Regression** (bài 2.2) học được. Vùng mờ quanh đường là vùng tin cậy. Tương quan 0,68 giữa hóa đơn và tiền boa: khá mạnh.

6.14.3. `lineplot`: xu hướng có gộp dữ liệu

```
fig, ax = plt.subplots(figsize=(8, 4))
sns.lineplot(data=df, x="tuoi_nha", y="gia_nha", hue="loai_nha", ax=ax)   # mỗi
tuổi nhà: vẽ TRUNG BÌNH + vùng tin cậy
ax.set_title("Giá trung bình theo tuổi nhà")
plt.show()
```

Bạn sẽ thấy: hai đoạn đường (Mới: 1–29 năm, Cũ: 30–52 năm) kèm dải mờ. Nhà rất mới (2–4 năm) hơi đắt, sau đó giá dao động quanh 180.000–220.000 rồi **vọt lên** gần 280.000 ở 52 năm. Dải mờ rộng ở hai đầu vì ở đó có ít khu dân cư (ít dữ liệu → kém chắc chắn). Cột 52 năm gộp tất cả khu "từ 52 năm trở lên" (giới hạn trần), gồm nhiều khu trung tâm đô thị cũ đắt đỏ.

Khi có nhiều dòng cùng giá trị `x`, `lineplot` tự gộp thành trung bình và vẽ khoảng tin cậy. Rất tiện cho dữ liệu thời gian có nhiều bản ghi mỗi ngày.

6.15. Seaborn: heatmap

6.15.1. Heatmap tương quan

```
cot = ["tuoi_nha", "tong_phong", "dan_so", "so_ho", "thu_nhap", "phong_moi_ho",
"gia_nha"]
tq = df[cot].corr()                              # bảng hệ số tương quan (sẽ học
kỹ ở bài 1.7)

fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(tq,
            annot=True, fmt=".2f",               # ghi số lên từng ô, 2 số lẻ
            cmap="coolwarm",                     # bảng màu PHÂN KỲ: xanh = âm, đỏ
= dương
            vmin=-1, vmax=1,                     # thang màu cố định từ -1 đến 1
            mask=np.triu(np.ones_like(tq, dtype=bool)),   # che nửa trên (trùng
lặp với nửa dưới)
            ax=ax)
ax.set_title("Thu nhập tương quan mạnh nhất với giá nhà")
plt.show()
print(tq["gia_nha"].round(2).sort_values(ascending=False))
```

Kết quả mẫu:

```
gia_nha         1.00
thu_nhap        0.69
phong_moi_ho    0.15
tong_phong      0.13
tuoi_nha        0.11
so_ho           0.06
dan_so         -0.03
Name: gia_nha, dtype: float64
```

- Ô `thu_nhap`–`gia_nha` đỏ đậm nhất (0,69). Khớp với scatter ở mục 6.3.
- Bộ ba `tong_phong`, `dan_so`, `so_ho` đỏ rực với nhau (≥ 0,86): cả ba đều đo **quy mô khu**, nói gần cùng một điều (đa cộng tuyến, sẽ học ở bài 2.6).
- `mask=np.triu(...)`: bảng tương quan đối xứng, che nửa trên cho đỡ rối mắt.

6.15.2. Heatmap từ pivot table

```
df["nhom_thu_nhap"] = pd.cut(df["thu_nhap"], bins=[0, 2, 4, 6, 8, 16],
                             labels=["<2", "2-4", "4-6", "6-8", "8+"])
df["nhom_tuoi"] = pd.cut(df["tuoi_nha"], bins=[0, 10, 20, 30, 40, 52],
                         labels=["0-10", "11-20", "21-30", "31-40", "41-52"])
bang = df.pivot_table(values="gia_nha", index="nhom_thu_nhap",
columns="nhom_tuoi",
                      aggfunc="median", observed=True) / 1000        # đổi sang
nghìn USD
```

```
fig, ax = plt.subplots(figsize=(8, 5))
sns.heatmap(bang, annot=True, fmt=".0f", cmap="YlOrRd", ax=ax)       # bảng màu
TUẦN TỰ: vàng → đỏ
ax.set_title("Giá trung vị (nghìn USD) theo thu nhập × tuổi nhà")
plt.show()
```

Bạn sẽ thấy: màu đậm dần **từ trên xuống** (thu nhập tăng → giá tăng mạnh). Theo chiều ngang (tuổi nhà) màu thay đổi ít hơn, nhưng ở các hàng thu nhập cao, cột nhà cũ (41–52) đậm hơn rõ. Heatmap kiểu này giúp đọc một bảng pivot 25 ô trong vài giây. Để ý các ô ghi đúng **500**: trung vị chạm giới hạn trần, nghĩa là quá nửa số khu trong ô đó bị ghi 500.001. Giá thật của chúng ta không biết.

6.16. Seaborn: nhìn nhiều biến cùng lúc

6.16.1. `pairplot`: mọi cặp biến

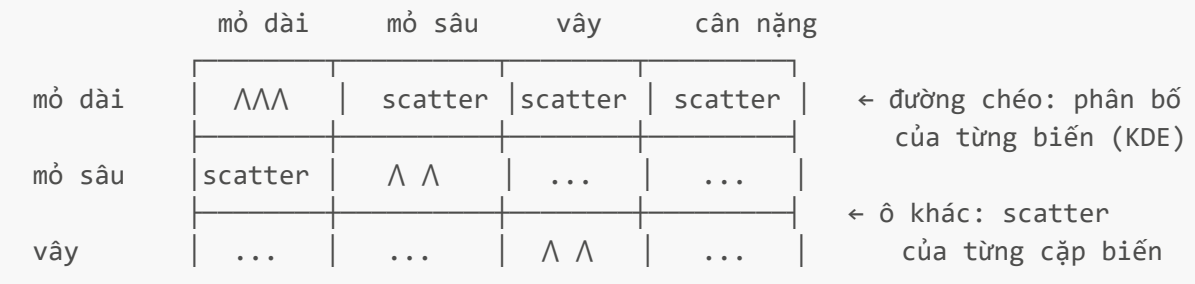
Dataset penguins (chim cánh cụt): 344 con thuộc 3 loài ở quần đảo Palmer (Nam Cực), do Tiến sĩ Kristen Gorman thu thập. Cột: loài, đảo, chiều dài mỏ, độ sâu (độ dày) mỏ, chiều dài vây, cân nặng, giới tính.

```
peng = sns.load_dataset("penguins").dropna()      # bỏ các dòng thiếu dữ liệu (bài
1.6)
print(peng.shape)
print(peng["species"].value_counts())

g = sns.pairplot(peng, hue="species",
                 vars=["bill_length_mm", "bill_depth_mm", "flipper_length_mm",
"body_mass_g"],
                 height=2.2, plot_kws={"s": 15, "alpha": 0.7})
g.figure.suptitle("Ba loài chim cánh cụt tách nhau khá rõ", y=1.02)
plt.show()
```

Kết quả mẫu:

```
(333, 7)
species
Adelie       146
Gentoo       119
Chinstrap     68
Name: count, dtype: int64
```



```
                ├──────────┼──────────┼──────────┼──────────┤
   cân nặng     │   ...    │   ...    │   ...    │  ∧  ∧    │
                └──────────┴──────────┴──────────┴──────────┘
```

Bạn sẽ thấy: 4 × 4 = 16 ô. Ba màu (ba loài) tách thành ba cụm rõ ràng ở nhiều ô, đặc biệt ô "mỏ dài – mỏ sâu". Loài Gentoo (màu xanh lá) nặng và vây dài hẳn hai loài kia.

Liên hệ ML: nếu các nhóm tách nhau rõ trên biểu đồ, mô hình **phân loại** (bài 2.4, 2.7, 2.9) sẽ dễ phân biệt chúng. Pairplot là bước khám phá rất hay trước khi chọn đặc trưng. Chú ý: với nhiều cột, pairplot vẽ rất chậm (10 cột = 100 ô). Chỉ chọn 4–6 cột quan trọng.

6.16.2. Figure-level: chia ô theo nhóm bằng `col`

```
g = sns.displot(data=tips, x="total_bill", col="time", row="smoker",   # mỗi tổ
hợp time × smoker một ô
                bins=20, height=3, aspect=1.3)
plt.show()

g = sns.catplot(data=tips, x="day", y="tip", col="sex", kind="box",    # kind:
loại biểu đồ (box, violin, bar, strip...)
                order=["Thur", "Fri", "Sat", "Sun"], height=3.5)
plt.show()

g = sns.relplot(data=tips, x="total_bill", y="tip", col="time", hue="smoker",
height=3.5)
plt.show()
```

Bạn sẽ thấy: hình 1 là lưới 2 × 2 histogram (hút thuốc / không × trưa / tối); hình 2 là 2 ô boxplot cạnh nhau (Nam / Nữ); hình 3 là 2 ô scatter (trưa / tối). Các ô **dùng chung thang trục**, nên so sánh rất công bằng.

Hàm figure-level	Tương ứng axes-level (`kind=`)
`sns.displot`	`histplot`, `kdeplot`, `ecdfplot`
`sns.catplot`	`boxplot`, `violinplot`, `barplot`, `countplot`, `stripplot`
`sns.relplot`	`scatterplot`, `lineplot`

6.16.3. `jointplot`: scatter + phân bố hai bên

```
g = sns.jointplot(data=peng, x="flipper_length_mm", y="body_mass_g",
hue="species", height=5)
plt.show()
```

Bạn sẽ thấy: một scatter ở giữa; phía trên là phân bố chiều dài vây, bên phải là phân bố cân nặng, cho từng loài.

6.17. Vẽ "bản đồ" từ kinh độ, vĩ độ và kiểm chứng giả thuyết

Ở bài 1.4 có câu hỏi: "*Vì sao khu nhà Cũ lại đắt hơn?*" Giả thuyết: nhà cũ ở trung tâm thành phố. Hãy để dữ liệu trả lời.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

diem = axes[0].scatter(df["kinh_do"], df["vi_do"],
                       c=df["gia_nha"],          # màu theo giá nhà
                       cmap="viridis",           # bảng màu tuần tự: tím (rẻ) →
vàng (đắt)
                       s=df["dan_so"] / 300,     # kích thước theo dân số
                       alpha=0.4)
fig.colorbar(diem, ax=axes[0], label="Giá nhà (USD)")
axes[0].set_title("Giá nhà trên bản đồ California")

diem2 = axes[1].scatter(df["kinh_do"], df["vi_do"], c=df["tuoi_nha"],
cmap="magma", s=2, alpha=0.4)
fig.colorbar(diem2, ax=axes[1], label="Tuổi nhà (năm)")
axes[1].set_title("Tuổi nhà trên bản đồ California")

for ax in axes:
    ax.set_xlabel("Kinh độ"); ax.set_ylabel("Vĩ độ")
    ax.annotate("San Francisco", xy=(-122.4, 37.8), xytext=(-124, 36.5),
arrowprops=dict(arrowstyle="->"))
    ax.annotate("Los Angeles", xy=(-118.25, 34.05), xytext=(-121, 33),
arrowprops=dict(arrowstyle="->"))
plt.tight_layout()
plt.show()
```

Bạn sẽ thấy: hình dáng bang California hiện ra chỉ từ các chấm. Bản đồ bên trái: vùng **vàng (đắt)** tập trung **dọc bờ biển**, quanh hai khu đô thị lớn San Francisco (vĩ độ ~37,8) và Los Angeles (vĩ độ ~34). Vùng nội địa phía đông chủ yếu màu tím (rẻ). Bản đồ bên phải: đúng hai khu đô thị đó có màu **sáng (nhà cũ)**, còn vùng ven và nội địa tối hơn (nhà mới).

Kiểm chứng giả thuyết bằng con số:

```
# Khoanh vùng gần đúng trung tâm 2 thành phố lớn bằng kinh độ, vĩ độ
la = df["vi_do"].between(33.9, 34.2) & df["kinh_do"].between(-118.5, -118.1)
sf = df["vi_do"].between(37.6, 37.9) & df["kinh_do"].between(-122.55, -122.2)
df["vung"] = np.select([la, sf], ["Trung tâm LA", "Trung tâm SF"], default="Nơi
khác")

print(df.groupby("vung").agg(so_khu=("gia_nha", "size"),
                             tuoi_tb=("tuoi_nha", "mean"),
                             gia_trung_vi=("gia_nha", "median"),
                             ti_le_nha_cu=("loai_nha", lambda s: (s ==
"Cũ").mean())).round(2))
```

Kết quả mẫu:

```
              so_khu  tuoi_tb  gia_trung_vi  ti_le_nha_cu
vung
Nơi khác       13486    25.86      170600.0          0.39
Trung tâm LA    2554    36.81      200000.0          0.79
Trung tâm SF     960    45.03      274000.0          0.90
```

Đúng như giả thuyết: nhà ở trung tâm LA và SF **cũ hơn** và **đắt hơn** phần còn lại của bang rất nhiều. Vậy **không phải "cũ" làm nhà đắt**, mà **vị trí** làm nhà vừa cũ vừa đắt.

> 💡 **Tương quan không phải nhân quả.** Hai thứ đi cùng nhau chưa chắc cái này gây ra cái kia. Có thể có một yếu tố thứ ba (ở đây là vị trí) đứng sau cả hai. Đây là một trong những bài học quan trọng nhất của khoa học dữ liệu.

Bạn cũng vừa thấy sức mạnh của **kết hợp**: Seaborn/Matplotlib để **thấy** mẫu hình, Pandas để **đo** nó bằng con số.

6.18. Checklist một biểu đồ tốt ✅

- ☐ **Tiêu đề** nói lên điều biểu đồ muốn khẳng định (ví dụ "Thu nhập càng cao, giá nhà càng cao"), không chỉ là "Biểu đồ 1".
- ☐ **Nhãn trục** kèm **đơn vị** (USD, m^2, năm...). Số lớn có dấu ngăn nghìn.
- ☐ Biểu đồ cột: trục y **bắt đầu từ 0**.
- ☐ Dữ liệu nhiều điểm thì dùng `alpha` nhỏ, `s` nhỏ, hoặc lấy mẫu.
- ☐ Chọn đúng loại biểu đồ theo bảng ở mục 5.3; đúng loại bảng màu theo mục 5.4.
- ☐ Nhóm có thứ tự (thứ trong tuần, nhóm tuổi) thì sắp đúng thứ tự (`order=`); nhóm không thứ tự thì sắp theo giá trị.
- ☐ Không dùng 3D, không dùng quá nhiều màu, không nhồi quá 3–4 thông tin vào một hình.

---

7. ⚠️ Lỗi thường gặp và hiểu lầm phổ biến

Lỗi 1: Trục y bị cắt làm phóng đại chênh lệch

```
ty_le = pd.Series({"Năm 2024": 96.2, "Năm 2025": 97.1})    # tỉ lệ khách hài lòng
(%)
fig, axes = plt.subplots(1, 2, figsize=(10, 3.5))
axes[0].bar(ty_le.index, ty_le.values, color=["gray", "tab:red"])
axes[0].set_ylim(96, 97.2)                                  # cắt trục: bắt đầu từ
96
axes[0].set_title("Trục cắt: tưởng tăng gấp 4 lần!")
axes[1].bar(ty_le.index, ty_le.values, color=["gray", "tab:red"])
axes[1].set_ylim(0, 100)                                    # trục đầy đủ từ 0
axes[1].set_title("Trục từ 0: chỉ tăng 0,9 điểm %")
plt.show()
```

Bạn sẽ thấy: hình trái, cột 2025 cao gấp mấy lần cột 2024. Hình phải, hai cột gần như bằng nhau.

- **Nhận biết:** nhìn số đầu tiên trên trục y của biểu đồ **cột**. Không phải 0 → cảnh giác.
- **Sửa:** biểu đồ cột luôn bắt đầu từ 0. Muốn nhấn mạnh thay đổi nhỏ, dùng biểu đồ đường hoặc ghi rõ con số chênh lệch. (Biểu đồ đường và scatter **không bắt buộc** bắt đầu từ 0.)

Lỗi 2: Overplotting (chồng điểm)

- **Nhận biết:** scatter thành một mảng màu đặc, không thấy đâu là chỗ đông.
- **Sửa:** giảm `s`, giảm `alpha` (mục 6.3.1); lấy mẫu `df.sample(2000)`; hoặc dùng biểu đồ mật độ 2 chiều `ax.hexbin(x, y, gridsize=50)` hay `sns.kdeplot(x=..., y=...)`.

Lỗi 3: Dùng `ax=` với hàm figure-level

```
fig, ax = plt.subplots()
sns.displot(data=tips, x="tip", ax=ax)       # displot tự tạo hình mới, KHÔNG nhận ax
plt.show()
```

Bạn sẽ thấy: một hình **trống** (của `plt.subplots()`), một hình histogram riêng, và dòng cảnh báo màu hồng:

```
UserWarning: `displot` is a figure-level function and does not accept the ax= parameter. You may wish to try histplot.
```

- **Nhận biết:** cảnh báo như trên; hoặc xuất hiện **một hình trống** cùng một hình khác bạn không muốn.
- **Sửa:** muốn ghép vào `subplots` của bạn → dùng hàm **axes-level** (`histplot`, `boxplot`, `scatterplot`...) với `ax=axes[i]`. Muốn chia ô theo nhóm → dùng figure-level với `col=`, đừng tự tạo `subplots`.

Lỗi 4: `barplot` tưởng là đếm, `countplot` tưởng là trung bình

- **Hiểu lầm:** "Cột của `sns.barplot(x="day", y="tip")` là tổng tiền boa mỗi ngày." Thực ra là **trung bình**.
- **Nhận biết:** con số trên trục y nhỏ bất thường so với kỳ vọng "tổng".
- **Sửa:** muốn tổng → `estimator="sum"`; muốn đếm → `countplot`. Luôn kiểm chứng bằng `groupby` (như mục 6.13.2).

Lỗi 5: Nhãn trục chồng chéo, không đọc được

```
quan = ["Quận 1", "Quận 3", "Quận 5", "Quận 7", "Quận 10", "Bình Thạnh", "Phú Nhuận", "Gò Vấp", "Tân Bình", "Thủ Đức"]
gia_m2 = [250, 180, 140, 120, 150, 110, 135, 85, 115, 75]
fig, axes = plt.subplots(1, 2, figsize=(13, 3.5))
axes[0].bar(quan, gia_m2)
axes[0].set_title("Nhãn đè lên nhau")
axes[1].bar(quan, gia_m2)
axes[1].tick_params(axis="x", rotation=45)          # sửa: xoay nhãn
```

```
axes[1].set_title("Xoay nhãn 45°")
plt.tight_layout()
plt.show()
```

Bạn sẽ thấy: hình trái các tên quận dính chùm vào nhau; hình phải các tên xoay nghiêng, đọc được. Cách tốt hơn nữa: dùng `barh` (mục 6.4.2).

Lỗi 6: Vẽ chồng lên hình cũ / quên `plt.show()`

- **Nhận biết:** trong file `.py` chạy ngoài Colab, không thấy hình nào hiện ra; hoặc biểu đồ mới vẽ đè lên biểu đồ cũ.
- **Vì sao:** Colab tự hiện hình ở cuối mỗi ô, còn script `.py` thì không. Gọi `plt.plot` nhiều lần mà không tạo hình mới thì tất cả vẽ vào **cùng một hình**.
- **Sửa:** luôn bắt đầu bằng `fig, ax = plt.subplots()` và kết thúc bằng `plt.show()`.

Lỗi 7: Tin vào đường KDE / đường xu hướng mà không nhìn điểm thật

- **Hiểu lầm:** "Đường `regplot` đi lên nên chắc chắn có quan hệ tuyến tính."
- **Nhận biết:** nhớ lại bộ tứ Anscombe: cả 4 bộ có **cùng một đường thẳng** khớp nhất!
- **Sửa:** luôn nhìn **điểm thật** (scatter, stripplot) bên cạnh đường làm mượt.

---

8. 🌎 Ứng dụng trong thực tế

8.1. Huấn luyện mô hình AI: đọc biểu đồ loss để phát hiện overfitting

```
vong = np.arange(1, 31)                                    # 30 vòng huấn luyện
(epoch)
loss_train = 2.0 * np.exp(-vong / 6) + 0.10                # loss trên dữ liệu
huấn luyện: giảm đều
loss_val = 2.0 * np.exp(-vong / 6) + 0.15 + 0.0009 * (vong - 8).clip(0) ** 2   #
loss kiểm tra: giảm rồi TĂNG lại

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(vong, loss_train, label="Train loss")
ax.plot(vong, loss_val, label="Validation loss")
tot_nhat = vong[np.argmin(loss_val)]
ax.axvline(tot_nhat, color="gray", linestyle="--", label=f"Dừng tốt nhất: vòng
{tot_nhat}")
ax.set_xlabel("Vòng huấn luyện (epoch)")
ax.set_ylabel("Loss (càng thấp càng tốt)")
ax.set_title("Sau vòng tối ưu, mô hình bắt đầu 'học vẹt'")
ax.legend()
plt.show()
print("Vòng tốt nhất:", tot_nhat)
```

Kết quả mẫu:

```
Vòng tốt nhất: 18
```

(Số liệu ở đây là mô phỏng.) Khi đường **validation** đi lên trong khi **train** vẫn giảm, mô hình đang **overfitting** (học thuộc dữ liệu huấn luyện, sẽ học kỹ ở bài 2.6 và 3.9). Kỹ sư AI nhìn biểu đồ này hằng ngày để quyết định khi nào dừng huấn luyện (**Early Stopping**).

8.2. Y tế: so sánh hiệu quả điều trị bằng boxplot

Bệnh viện so sánh thời gian nằm viện của bệnh nhân giữa 3 phác đồ điều trị. Một boxplot cho thấy ngay phác đồ nào có trung vị thấp nhất, phác đồ nào có nhiều ca bất thường (outlier) cần xem xét. Bảng trung bình đơn thuần có thể bị vài ca nằm viện rất lâu làm sai lệch.

8.3. Thương mại điện tử: dashboard bán hàng

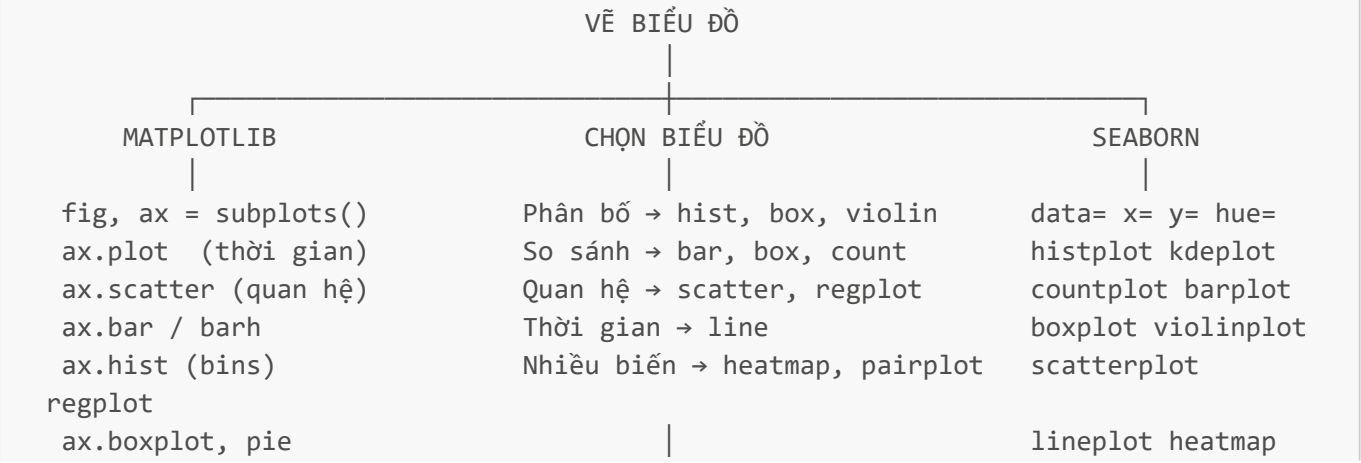
Một dashboard bán hàng điển hình gồm 4 ô `subplots`: **line** doanh thu theo ngày (xu hướng), **barh** top 10 sản phẩm (xếp hạng), **heatmap** đơn hàng theo giờ × thứ trong tuần (khung giờ vàng để chạy quảng cáo), **histogram** giá trị đơn hàng (đa số khách chi bao nhiêu). Bạn sẽ dựng dashboard tương tác thật bằng Streamlit ở Phần C.

---

9. 📝 Tóm tắt

1. **Luôn vẽ dữ liệu trước khi làm gì khác**: thống kê giống nhau có thể là dữ liệu khác hẳn (Anscombe).
2. Matplotlib: `fig, ax = plt.subplots()` → `ax.plot/scatter/bar/hist/boxplot` → `ax.set_title/xlabel/ylabel` → `plt.show()`.
3. Chọn biểu đồ theo câu hỏi: phân bố → histogram/box; so sánh nhóm → bar/box/violin; quan hệ → scatter/regplot; thời gian → line; nhiều biến → heatmap/pairplot.
4. Seaborn nói chung một "ngôn ngữ": `data=`, `x=`, `y=`, `hue=`; figure-level thêm `col=`, `row=`; axes-level nhận `ax=`.
5. Scatter nhiều điểm: giảm `s`, `alpha`. Histogram: thử nhiều `bins`. Cột: trục từ 0.
6. Màu: tuần tự cho giá trị thấp→cao, phân kỳ cho tương quan, định tính cho nhóm. Tránh `jet`, 3D, pie nhiều lát.
7. Biểu đồ giúp **thấy** mẫu hình, Pandas giúp **đo** nó; tương quan không phải nhân quả.

Sơ đồ tư duy:



```
 axes[i, j], suptitle            ĐỌC & CẢNH GIÁC              pairplot jointplot
 set_title/xlabel/ylabel         trần, outlier, lệch
displot/catplot/relplot (col=)
 legend, grid, annotate          trục cắt, chồng điểm                  |
 savefig(dpi, bbox)              tương quan ≠ nhân quả        axes-level: ax= ✓
                                                              figure-level: ax=
X
```

---

10. ✅ Trắc nghiệm

> Tự làm hết 10 câu trước, rồi mới xem mục 🔒 **Đáp án trắc nghiệm** ở cuối bài.

Mức 1 – Nhớ / Hiểu

Câu 1. Trong Matplotlib, `Axes` là gì?

- A. Trục x và trục y.
- B. Một ô biểu đồ nằm trên Figure.
- C. Toàn bộ tờ giấy.
- D. Hộp chú thích.

Câu 2. Bạn muốn biết điểm thi của 10.000 thí sinh phân bố thế nào (đa số ở đâu, lệch không). Biểu đồ phù hợp nhất?

- A. Pie chart
- B. Line chart
- C. Histogram
- D. Heatmap

Câu 3. Tham số `alpha=0.2` trong `ax.scatter` có tác dụng gì?

- A. Giảm kích thước điểm xuống 20%.
- B. Làm điểm trong suốt, giúp thấy chỗ điểm dày đặc.
- C. Chỉ vẽ 20% số điểm.
- D. Làm mượt dữ liệu.

Câu 4. Để hiển thị bảng hệ số tương quan (từ −1 đến 1) bằng heatmap, bảng màu nào hợp lý nhất?

- A. `"viridis"` (tuần tự)
- B. `"coolwarm"` (phân kỳ) với `vmin=-1, vmax=1`
- C. `"tab10"` (định tính)
- D. `"jet"` (cầu vồng)

Mức 2 – Vận dụng

Câu 5. Sau lệnh `fig, axes = plt.subplots(2, 3)`, ô ở **hàng thứ hai, cột thứ nhất** là gì?

- A. `axes[2, 1]`
- B. `axes[1, 0]`
- C. `axes[0, 1]`

- D. `axes[1]`

Câu 6. Đoạn code sau vẽ gì?

```
sns.barplot(data=tips, x="day", y="total_bill")
```

- A. Số hóa đơn mỗi ngày.
- B. Tổng tiền hóa đơn mỗi ngày.
- C. Tiền hóa đơn trung bình mỗi ngày, kèm vạch khoảng tin cậy.
- D. Hóa đơn lớn nhất mỗi ngày.

Câu 7. Bạn muốn vẽ 3 histogram giá nhà, mỗi ô cho một nhóm thu nhập, các ô **dùng chung thang trục**. Cách nào gọn nhất?

- A. `sns.histplot(data=df, x="gia_nha", hue="nhom_thu_nhap")`
- B. `sns.displot(data=df, x="gia_nha", col="nhom_thu_nhap")`
- C. `sns.heatmap(df[["gia_nha", "nhom_thu_nhap"]])`
- D. `sns.pairplot(df, hue="nhom_thu_nhap")`

Câu 8. Đoạn code sau có vấn đề gì?

```
fig, ax = plt.subplots()
ax.bar(["Q1", "Q2"], [1020, 1050])
ax.set_ylim(1000, 1060)
ax.set_title("Doanh thu tăng vọt!")
```

- A. Không có vấn đề.
- B. Thiếu `plt.show()` nên lỗi.
- C. Trục y không bắt đầu từ 0 nên phóng đại mức tăng 3% thành "tăng vọt".
- D. `ax.bar` không nhận chữ ở trục x.

Mức 3 – Phân tích (bẫy hay gặp)

Câu 9. Bạn vẽ histogram giá nhà và thấy một cột cao vọt bất thường ở **đúng giá trị lớn nhất** (500.001). Giải thích nào **hợp lý nhất**?

- A. Có rất nhiều nhà giá đúng 500.001 USD vì đó là giá phổ biến.
- B. Dữ liệu bị giới hạn trần: mọi giá lớn hơn đều được ghi thành 500.001.
- C. Số `bins` quá ít.
- D. Lỗi của Matplotlib.

Câu 10. Một bạn vẽ heatmap tương quan, thấy `kinh_do`–`gia_nha` có hệ số gần 0 và kết luận: "Vị trí không ảnh hưởng giá nhà, bỏ cột kinh độ". Nhận định nào đúng?

- A. Đúng, hệ số gần 0 nghĩa là không liên quan.
- B. Sai: hệ số tương quan chỉ đo quan hệ **đường thẳng**; bản đồ ở mục 6.17 cho thấy vị trí ảnh hưởng giá rất mạnh nhưng theo kiểu phức tạp (gần biển, gần đô thị).

- C. Sai: phải dùng `cmap="viridis"` mới đọc đúng.
- D. Đúng, nhưng chỉ khi dữ liệu không có outlier.

---

11. 🛠️ Thực hành

Bài 1 (Cơ bản): Khám phá dữ liệu tiền boa

Dùng dataset `tips` (mục 6.11). Làm theo từng bước.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")
tips = sns.load_dataset("tips")

# Bước 1: Tạo cột ti_le_boa = tip / total_bill * 100 (phần trăm).
# TODO

# Bước 2: Vẽ histogram của ti_le_boa với 30 bins, thêm đường KDE.
#         Đặt tiêu đề và nhãn trục có đơn vị. (Gợi ý: sns.histplot(..., kde=True))
# TODO

# Bước 3: Có hóa đơn nào có tỉ lệ boa bất thường (> 40%) không? In các dòng đó.
# TODO

# Bước 4: Vẽ boxplot ti_le_boa theo "time" (Lunch/Dinner), tách màu theo "smoker".
#         (Gợi ý: sns.boxplot(data=..., x="time", y="ti_le_boa", hue="smoker"))
# TODO

# Bước 5: Vẽ countplot số hóa đơn theo "size" (số người trong bàn).
# TODO

# Bước 6: Vẽ scatter total_bill vs tip, màu theo "size". Thêm một đường thẳng tham
chiếu
#         "boa 15%": y = 0.15 * x. (Gợi ý: ax.plot(x, 0.15 * x, "--"))
# TODO

# Bước 7: Gom 4 biểu đồ ở bước 2, 4, 5, 6 vào MỘT hình 2×2 có suptitle. Lưu thành
"tips_dashboard.png".
#         (Gợi ý: fig, axes = plt.subplots(2, 2, ...); truyền ax=axes[i, j] cho
hàm Seaborn)
# TODO
```

Bài 2 (Trung bình): Chim cánh cụt Palmer

Dataset: Palmer Penguins: 344 con chim cánh cụt thuộc 3 loài (Adelie, Chinstrap, Gentoo) ở quần đảo Palmer, Nam Cực, thu thập bởi Tiến sĩ Kristen Gorman và Trạm nghiên cứu Palmer (LTER). Nạp bằng `sns.load_dataset("penguins")`.

Yêu cầu:

1. Đếm số con mỗi loài trên mỗi đảo bằng biểu đồ. Loài nào chỉ sống trên một đảo?
2. So sánh cân nặng giữa 3 loài, tách theo giới tính. Chọn loại biểu đồ phù hợp và giải thích vì sao bạn chọn.
3. Vẽ scatter chiều dài mỏ (`bill_length_mm`) vs độ sâu mỏ (`bill_depth_mm`) **không** tô màu. Tính hệ số tương quan. Rồi vẽ lại **có** tô màu theo loài và tính tương quan **trong từng loài**. Bạn thấy điều gì bất ngờ?
4. Vẽ heatmap tương quan của 4 cột số.
5. Viết 3 câu kết luận, mỗi câu dựa trên một biểu đồ bạn đã vẽ.

Gợi ý: `countplot` với `hue`; `boxplot`/`violinplot`; `scatterplot`, `regplot`; `groupby("species")[[...]].corr()`; `dropna()`.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
peng = sns.load_dataset("penguins")
# TODO
```

Bài 3 (Thử thách): Một hình, một câu chuyện

Chọn **một** dataset: California (bài này), Titanic (bài 1.4, `sns.load_dataset("titanic")`) hoặc tips. Tự đặt **một câu hỏi** đáng quan tâm (ví dụ: "Điều gì quyết định cơ hội sống sót trên Titanic?", "Khu nào ở California đáng mua nhất?").

Thiết kế **một hình duy nhất** gồm 3–4 ô biểu đồ trả lời câu hỏi đó, như một slide bạn sẽ trình bày cho sếp.

```
# TODO: câu hỏi của bạn:
# TODO: code vẽ hình
```

Tiêu chí tự đánh giá:

- ☐ Hình có `suptitle` là **câu trả lời** cho câu hỏi, không phải tên dataset.
- ☐ Mỗi ô dùng **đúng loại biểu đồ** cho thông tin nó trình bày (mục 5.3), và có tiêu đề nói lên một phát hiện.
- ☐ Mọi trục có nhãn và đơn vị; số lớn có dấu ngăn nghìn; biểu đồ cột bắt đầu từ 0.
- ☐ Màu sắc nhất quán: cùng một nhóm mang cùng một màu ở mọi ô.
- ☐ Ít nhất một ô **kiểm chứng** hoặc **phản bác** một giả thuyết ban đầu của bạn (như mục 6.17).
- ☐ Hình được lưu ra file `.png` và một người không biết code xem vẫn hiểu.

---

12. 🎴 Flashcard

Mặt trước	Mặt sau
Bộ tứ Anscombe dạy điều gì?	Thống kê giống nhau vẫn có thể là dữ liệu khác hẳn. Luôn vẽ!
Figure vs Axes?	Figure là tờ giấy. Axes là một ô biểu đồ trên tờ giấy.
Phân bố / so sánh nhóm / quan hệ / thời gian dùng biểu đồ gì?	Histogram / boxplot, bar / scatter / line.
`sns.barplot` vẽ gì mặc định?	Trung bình của `y` mỗi nhóm + khoảng tin cậy. Đếm thì dùng `countplot`.
Axes-level vs figure-level Seaborn?	Axes-level (`histplot`...) nhận `ax=`. Figure-level (`displot`...) tự tạo hình, chia ô bằng `col=`.

13. 🚀 Đọc thêm

- Matplotlib – Quick start guide – có hình giải phẫu Figure/Axes chính thức.
- Seaborn – An introduction và thư viện ví dụ: xem hình, bấm vào lấy code.
- From Data to Viz – sơ đồ chọn biểu đồ theo kiểu dữ liệu, kèm cảnh báo lỗi thường gặp.
- *Fundamentals of Data Visualization*, Claus O. Wilke – sách đọc miễn phí tại clauswilke.com/dataviz.
- Python Graph Gallery – hàng trăm biểu đồ mẫu có code Python.

Câu hỏi gợi mở cho bài sau: Hôm nay bạn đã **nhìn thấy** đủ thứ "bẩn" trong dữ liệu: giới hạn trần, outlier, số phòng mỗi hộ vô lý, giá trị thiếu trong penguins. Nhưng thấy rồi thì **làm gì** với chúng? Xóa đi, sửa lại, hay giữ nguyên? Và cột chữ như "Cũ/Mới", "Quận 1/Quận 7" thì làm sao biến thành số cho mô hình? Đó là bài 1.6: Làm sạch dữ liệu.

🔒 Đáp án trắc nghiệm

> Mẹo Notion: chọn tiêu đề này → `Turn into → Toggle heading 2` để ẩn đáp án.

Câu	Đáp án	Giải thích
1	**B**	Axes là một ô biểu đồ (mục 5.1). A là bẫy chữ nghĩa: "trục" là **Axis**. C là Figure. D là Legend.
2	**C**	Histogram cho thấy phân bố một biến số. A dùng cho tỉ lệ vài phần. B dùng cho thời gian. D dùng cho bảng hai chiều.
3	**B**	`alpha` là độ trong suốt. A là việc của `s`. C phải dùng `df.sample`. D là việc của KDE.
4	**B**	Tương quan có điểm giữa ý nghĩa (0) và hai phía đối lập → bảng màu phân kỳ, cố định thang −1..1. A không phân biệt được âm/dương. C dành cho nhóm. D gây ảo giác màu.
5	**B**	Chỉ số bắt đầu từ 0: hàng thứ hai = 1, cột thứ nhất = 0. A vượt giới hạn. C là hàng 1, cột 2. D lấy cả hàng thứ hai (một mảng 3 ô).
6	**C**	`barplot` mặc định tính **trung bình** kèm khoảng tin cậy (mục 6.13.2, Lỗi 4). A là `countplot`. B cần `estimator="sum"`. D cần `estimator="max"`.

Câu	Đáp án	Giải thích
7	**B**	Figure-level `displot` với `col=` tạo mỗi nhóm một ô, chung thang trục. A vẽ chung một ô. C, D không phải histogram theo nhóm.
8	**C**	Lỗi 1: trục cắt. 1050/1020 chỉ tăng ~3% nhưng nhìn như gấp mấy lần. B: Colab tự hiện hình, thiếu `show` không gây lỗi. D sai: `bar` nhận chữ.
9	**B**	Dấu hiệu kinh điển của giới hạn trần (bài 1.4, mục 6.5.1). A: con số 500.001 quá "tròn" để là giá phổ biến tự nhiên. C: thay bins cột đó vẫn vọt lên. D vô căn cứ.
10	**B**	Hệ số tương quan Pearson chỉ đo quan hệ **tuyến tính** (giống bộ B của Anscombe). Giá cao ở **cả** SF (kinh độ −122) lẫn LA (−118), còn ở giữa là vùng rẻ, nên quan hệ không phải đường thẳng. A, D là bẫy. C: màu không đổi bản chất con số.

🔒 Lời giải thực hành

> Mẹo Notion: chọn tiêu đề này → `Turn into → Toggle heading 2`.

Lời giải Bài 1

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")
tips = sns.load_dataset("tips")

tips["ti_le_boa"] = tips["tip"] / tips["total_bill"] * 100                  # 1
print(tips["ti_le_boa"].describe().round(1))

print(tips[tips["ti_le_boa"] > 40][["total_bill", "tip", "size",
"ti_le_boa"]].round(1))   # 3

fig, axes = plt.subplots(2, 2, figsize=(13, 9))                             # 7
(gồm 2, 4, 5, 6)

sns.histplot(data=tips, x="ti_le_boa", bins=30, kde=True, ax=axes[0, 0])   # 2
axes[0, 0].set_title("Đa số khách boa 12–20% hóa đơn")
axes[0, 0].set_xlabel("Tỉ lệ boa (%)")

sns.boxplot(data=tips, x="time", y="ti_le_boa", hue="smoker", ax=axes[0, 1])   # 4
axes[0, 1].set_title("Tỉ lệ boa theo bữa và hút thuốc")
axes[0, 1].set_ylabel("Tỉ lệ boa (%)")

sns.countplot(data=tips, x="size", ax=axes[1, 0])                           # 5
axes[1, 0].set_title("Phần lớn là bàn 2 người")
axes[1, 0].set_xlabel("Số người trong bàn")

sns.scatterplot(data=tips, x="total_bill", y="tip", hue="size", palette="viridis",
```

```
ax=axes[1, 1])   # 6
x = np.linspace(0, tips["total_bill"].max(), 50)
axes[1, 1].plot(x, 0.15 * x, "--", color="red", label="Boa 15%")
axes[1, 1].legend()
axes[1, 1].set_title("Hóa đơn càng lớn, tiền boa càng nhiều")
axes[1, 1].set_xlabel("Hóa đơn (USD)"); axes[1, 1].set_ylabel("Tiền boa (USD)")

fig.suptitle("Khách nhà hàng boa trung bình khoảng 16% hóa đơn", fontsize=15)
plt.tight_layout()
fig.savefig("tips_dashboard.png", dpi=150, bbox_inches="tight")
plt.show()
```

Kết quả mẫu:

```
count    244.0
mean      16.1
std        6.1
min        3.6
25%       12.9
50%       15.5
75%       19.1
max       71.0
Name: ti_le_boa, dtype: float64
     total_bill  tip  size  ti_le_boa
172         7.2  5.2     2       71.0
178         9.6  4.0     2       41.7
```

Nhận xét: có 2 hóa đơn boa trên 40%, đều là hóa đơn **nhỏ** (dưới 10 USD), boa tròn số. Đó là outlier "thật", không phải lỗi. Đường đỏ 15% chạy giữa đám mây điểm: phần lớn khách boa quanh mức này. Tỉ lệ boa giữa các nhóm khá giống nhau; nhóm hút thuốc phân tán rộng hơn.

Lời giải Bài 2

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
peng = sns.load_dataset("penguins").dropna()
print(peng.shape)

fig, axes = plt.subplots(2, 2, figsize=(13, 10))

sns.countplot(data=peng, x="island", hue="species", ax=axes[0, 0])
# 1
axes[0, 0].set_title("Gentoo chỉ ở Biscoe, Chinstrap chỉ ở Dream")
print(pd.crosstab(peng["island"], peng["species"]))

sns.boxplot(data=peng, x="species", y="body_mass_g", hue="sex", ax=axes[0, 1])
# 2
```

```
axes[0, 1].set_title("Gentoo nặng nhất; con đực nặng hơn con cái ở cả 3 loài")
axes[0, 1].set_ylabel("Cân nặng (g)")

sns.regplot(data=peng, x="bill_length_mm", y="bill_depth_mm", ax=axes[1, 0],
# 3a: không tô màu
            scatter_kws={"s": 10, "color": "gray"}, line_kws={"color": "black"})
axes[1, 0].set_title("Gộp chung: mỏ càng dài càng... nông?")

sns.scatterplot(data=peng, x="bill_length_mm", y="bill_depth_mm", hue="species",
ax=axes[1, 1])   # 3b
for loai, nhom in peng.groupby("species"):
    he_so = np.polyfit(nhom["bill_length_mm"], nhom["bill_depth_mm"], 1)   # đường
thẳng khớp nhất (bài 2.2)
    xs = np.linspace(nhom["bill_length_mm"].min(), nhom["bill_length_mm"].max(),
10)
    axes[1, 1].plot(xs, np.polyval(he_so, xs), color="black", linewidth=1)
axes[1, 1].set_title("Tách theo loài: trong MỖI loài, mỏ dài thì sâu hơn")
plt.tight_layout()
plt.show()

print("Tương quan chung:",
peng["bill_length_mm"].corr(peng["bill_depth_mm"]).round(2))
print(peng.groupby("species")[["bill_length_mm",
"bill_depth_mm"]].corr().round(2).xs("bill_depth_mm", level=1)["bill_length_mm"])

cot_so = ["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]
# 4
fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(peng[cot_so].corr(), annot=True, fmt=".2f", cmap="coolwarm", vmin=-1,
vmax=1, ax=ax)
plt.show()
```

Kết quả mẫu:

```
(333, 7)
species    Adelie  Chinstrap  Gentoo
island
Biscoe         44          0     119
Dream          55         68       0
Torgersen      47          0       0
Tương quan chung: -0.23
species
Adelie       0.39
Chinstrap    0.65
Gentoo       0.65
Name: bill_length_mm, dtype: float64
```

Điều bất ngờ ở câu 3: Nghịch lý Simpson (Simpson's paradox).

- Gộp cả 3 loài, tương quan **âm** (−0,23): mỏ càng dài càng nông.

- Tách từng loài, tương quan đều **dương** (từ 0,39 đến 0,65): trong cùng một loài, mỏ dài thì sâu hơn.
- Lý do: loài Gentoo có mỏ **dài nhưng nông**, loài Adelie có mỏ **ngắn nhưng sâu**. Khi gộp lại, sự khác biệt **giữa các loài** lấn át xu hướng **trong từng loài**.
- Bài học: một biến ẩn (loài) có thể **đảo ngược** kết luận. Luôn thử `hue=` theo các nhóm quan trọng trước khi kết luận.

Ba kết luận mẫu: (1) Gentoo chỉ sống trên đảo Biscoe, Chinstrap chỉ trên đảo Dream; (2) Gentoo nặng hơn hẳn hai loài kia, và ở cả ba loài, con đực nặng hơn con cái; (3) chiều dài vây và cân nặng tương quan rất mạnh (0,87): con nào nặng thì vây dài.

Lời giải Bài 3 (mẫu với Titanic)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib.ticker as mtick
sns.set_theme(style="whitegrid")
tt = sns.load_dataset("titanic")
# Câu hỏi: Điều gì quyết định cơ hội sống sót trên Titanic?

mau_gioi = {"male": "tab:blue", "female": "tab:red"}         # màu nhất quán cho
mọi ô
fig, axes = plt.subplots(2, 2, figsize=(13, 9))

sns.barplot(data=tt, x="sex", y="survived", hue="sex", palette=mau_gioi,
errorbar=None, ax=axes[0, 0])
axes[0, 0].set_title("Nữ sống sót gấp gần 4 lần nam")
axes[0, 0].yaxis.set_major_formatter(mtick.PercentFormatter(1.0))   # 0.74 → 74%
axes[0, 0].set_ylim(0, 1)

sns.pointplot(data=tt, x="pclass", y="survived", hue="sex", palette=mau_gioi,
ax=axes[0, 1])
axes[0, 1].set_title("Hạng vé càng cao, cơ hội càng lớn, ở cả hai giới")
axes[0, 1].yaxis.set_major_formatter(mtick.PercentFormatter(1.0))
axes[0, 1].set_ylim(0, 1)

tt["nhom_tuoi"] = pd.cut(tt["age"], bins=[0, 12, 17, 59, 100], labels=["0-12",
"13-17", "18-59", "60+"])
sns.barplot(data=tt, x="nhom_tuoi", y="survived", errorbar=None, color="gray",
ax=axes[1, 0])
axes[1, 0].set_title("Trẻ em được ưu tiên: hơn 50% sống sót")
axes[1, 0].yaxis.set_major_formatter(mtick.PercentFormatter(1.0))
axes[1, 0].set_ylim(0, 1)

sns.histplot(data=tt, x="fare", hue="alive", bins=40, multiple="stack", ax=axes[1,
1])
axes[1, 1].set_xlim(0, 300)
axes[1, 1].set_title("Vé rẻ (< 20) chiếm đa số người tử nạn")
axes[1, 1].set_xlabel("Giá vé (bảng Anh)")

for ax in axes.flat:
```

```
    ax.set_ylabel(ax.get_ylabel().replace("survived", "Tỉ lệ sống sót"))
fig.suptitle("Trên Titanic, giới tính và hạng vé quyết định cơ hội sống sót",
fontsize=15)
plt.tight_layout()
fig.savefig("titanic_cau_chuyen.png", dpi=150, bbox_inches="tight")
plt.show()
print(tt.groupby("sex")["survived"].mean().round(2).to_dict())
```

Kết quả mẫu:

```
{'female': 0.74, 'male': 0.19}
```

Vì sao thiết kế như vậy: ô 1 trả lời "yếu tố mạnh nhất" (giới tính); ô 2 thêm yếu tố thứ hai và cho thấy nó đúng ở cả hai giới (`pointplot` vẽ trung bình từng nhóm nối bằng đường, kèm khoảng tin cậy); ô 3 kiểm chứng giả thuyết "phụ nữ **và trẻ em** trước"; ô 4 nhìn từ góc khác (giá vé) để củng cố câu chuyện về hạng vé. Màu nam/nữ giống nhau ở ô 1 và 2. Tiêu đề lớn là **câu trả lời**, không phải "Biểu đồ Titanic".